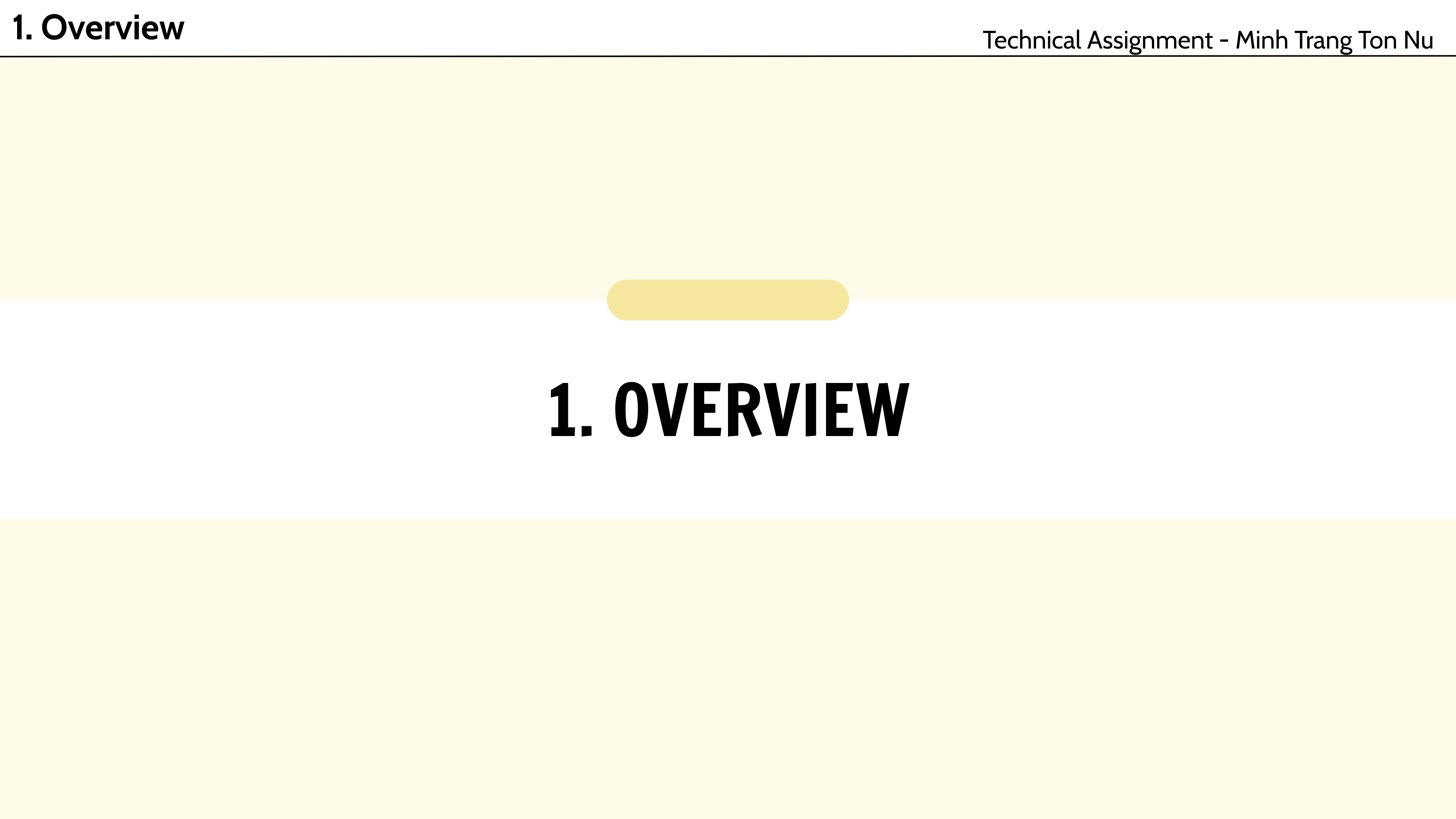


SINGLE 8 BIT TIMER

Minh Trang Ton Nu



1. OVERVIEW

8-bit Timer is a general-purpose, single-channel module with the key features:

- **8-bit Single-Channel Counter** Compact timer module featuring one 8-bit counter, suitable for general-purpose timing applications.
- **Selection of Four external clock sources** Offers four external clock inputs derived from the main system clock (PCLK)
 - + CLK_IN [0] (= PCLK/2)
 - + CLK_IN [1] (= PCLK/4)
 - + CLK_IN [2] (= PCLK/8)
 - + CLK_IN [3] (= PCLK/16)
- **Edge-Triggered Counting** Counter increments or decrements on each rising edge of the selected clock source.
- **Counter Initialization** Counter value can be loaded from Timer Data Register (TDR) when TCR[7] = 1 is set.
- **Counters can operate in different modes**
 - + Up Count: Increments by 1 on each clock cycle
 - + Down Count: Decrements by 1 on each clock cycle
- **Control and Status Registers** Configurable through control (TCR) and status (TSR) registers to manage and monitor timer behavior, such as enabling the counter, setting modes, or detecting overflow/underflow.

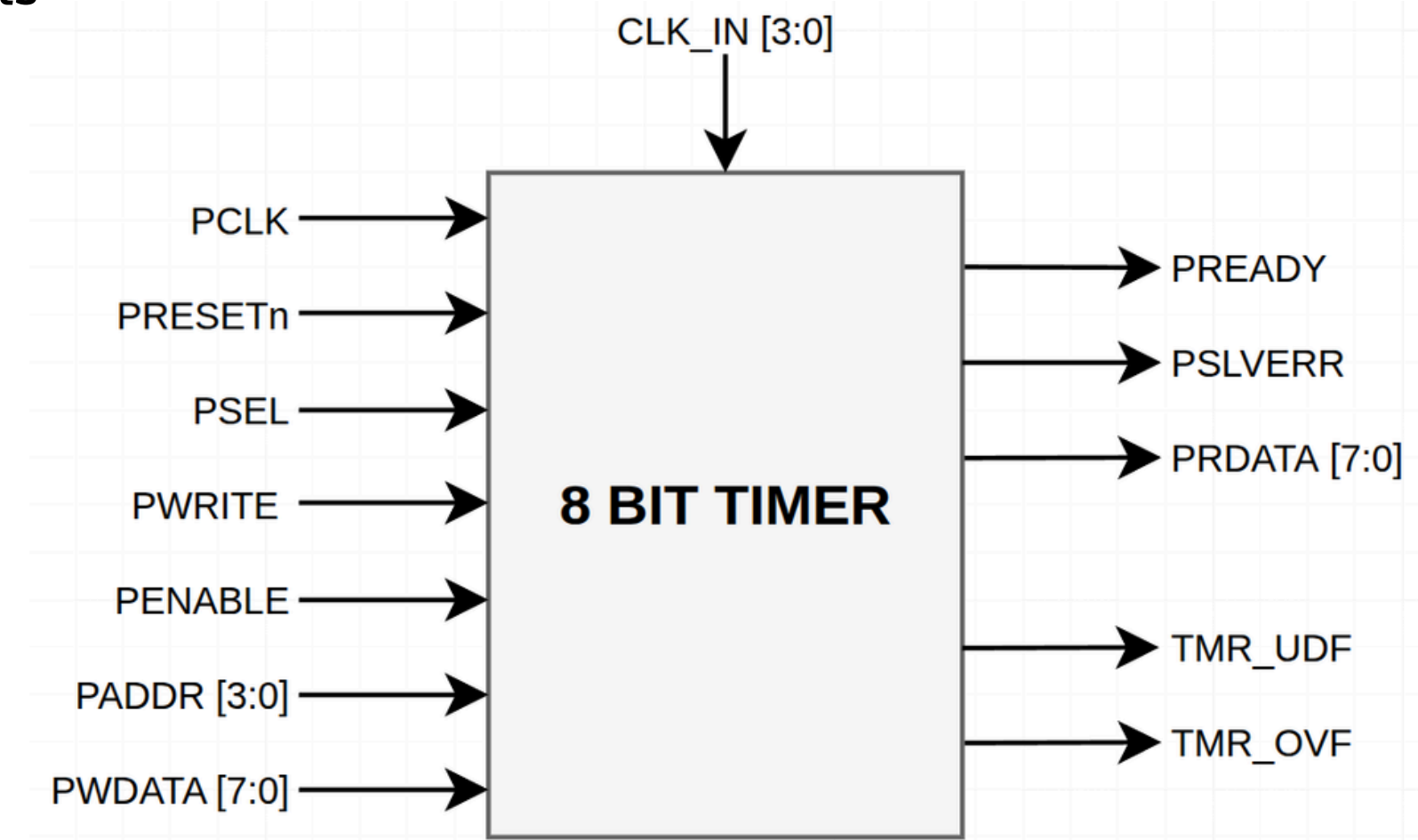


Figure 8-bit Timer Input/Output Pin

Table 1.1. TMR Input Pins Configuration

Port Name	Bit Width	I/O	Description
CLK_IN[3:0]	4	Input	Timer clock sources used for counting (selected by prescaler field in control register).
PCLK	1	Input	System clock for the APB interface.
PRESETn	1	Input	Asynchronous active-LOW reset for the timer and APB logic.
PSEL	1	Input	APB slave select; HIGH during an access to this timer IP.
PWRITE	1	Input	APB transfer direction: HIGH = write, LOW = read.
PENABLE	1	Input	Indicates second and subsequent cycles of an APB transfer (enable phase).
PADDR	3	Input	APB address bus used to select the internal registers (TDR, TCR, TSR, ...).
PWDATA	8	Input	APB write data bus; carries data to be written into the selected register.
PRDATA	8	Output	APB read data bus; returns data from the selected register.
PREADY	1	Output	APB ready signal from the slave; LOW can extend the transfer, HIGH completes the transfer.
PSLVERR	1	Output	Error indication for an APB transfer (e.g., invalid address or write to read-only register).
TMR_OVF	1	Output	Timer status flag: asserted when the counter counts up from 8'h00 to 8'hFF (overflow).
TMR_UDF	1	Output	Timer status flag: asserted when the counter counts down from 8'hFF to 8'h00 (underflow).

Table Configuration Registers

Offset	Register Name	Description	Bit Width	Access	Reset value
0x0	Timer Data Register TDR	Load value that will be transferred into TCNT	8	R/W	0
0x1	Timer Control Register TCR	Control bits for timer mode, enable, direction, prescale	8	R/W	0
0x2	Timer Status Register TSR	Status flags (overflow, underflow, etc.)	8	R/W	0
0x3	Timer Counter TCNT	Current 8-bit counter value	8	R	0

TCNT - Bit Description

Bit index	7	6	5	4	3	2	1	0
Bit name	—	—	—	—	—	—	—	—
Reset	0	0	0	0	0	0	0	0
R/W	R	R	R	R	R	R	R	R

If TCR[7] = 1, TCNT is loaded with value from TDR; otherwise, normal counting continues.

TDR – Bit Description

Bit index	7	6	5	4	3	2	1	0
Bit name	–	–	–	–	–	–	–	–
Reset	0	0	0	0	0	0	0	0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

TCR – Bit Description

The TCR is an 8-bit read/write register that controls the operation mode of the timer. Each bit has a specific function.

Bit	Bit Name	R/W	Reset	Description
TCR[7]	Load	R/W	0	When set to 1, the value in TDR is immediately loaded into TCNT. When 0, normal counting operation continues.
TCR[6]	Reserved	R	0	Reserved bit. Should always be written as 0.
TCR[5]	Up/Dw	R/W	0	Counter direction control: 0 – count up (increment); 1 – count down (decrement).
TCR[4]	En	R/W	0	Timer enable: 0 – disable the timer; 1 – enable the timer.
TCR[3:2]	Reserved	R	0	Reserved bits. Should always be written as 00.
TCR[1:0]	cks[1:0]	R/W	0	Clock select for internal timer clock: 00 – PCLK/2, 01 – PCLK/4, 10 – PCLK/8, 11 – PCLK/16.

TSR – Bit Description

Bit	Bit Name	R/W	Description
TSR[7:2]	Reserved	R	Reserved bits. Should always be written as 0.
TSR[1]	TMR_UDF	R/W*	Set to 1 when TCNT underflows from 8'h00 to 8'hFF in down-counting mode. This bit is set only by hardware and cleared by software.
TSR[0]	TMR_OVF	R/W*	Set to 1 when TCNT overflows from 8'hFF to 8'h00 in up-counting mode. This bit is set only by hardware and cleared by software.

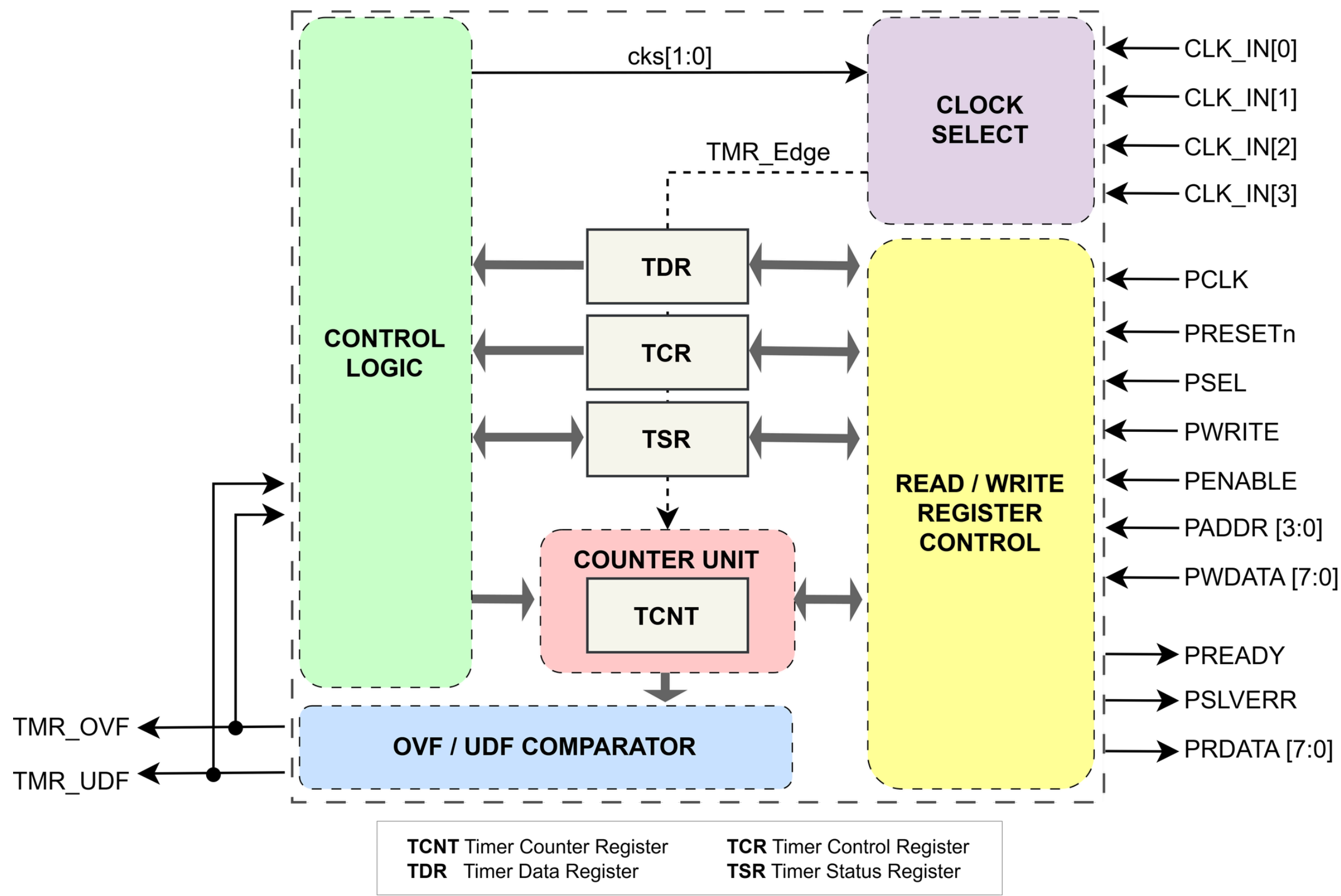
2. Design Specifications

2.1. Block Diagram

2.2. Sub-blocks (Architecture & RTL)

2.3. Top-level RTL

2.1. Block Diagram



Block diagram

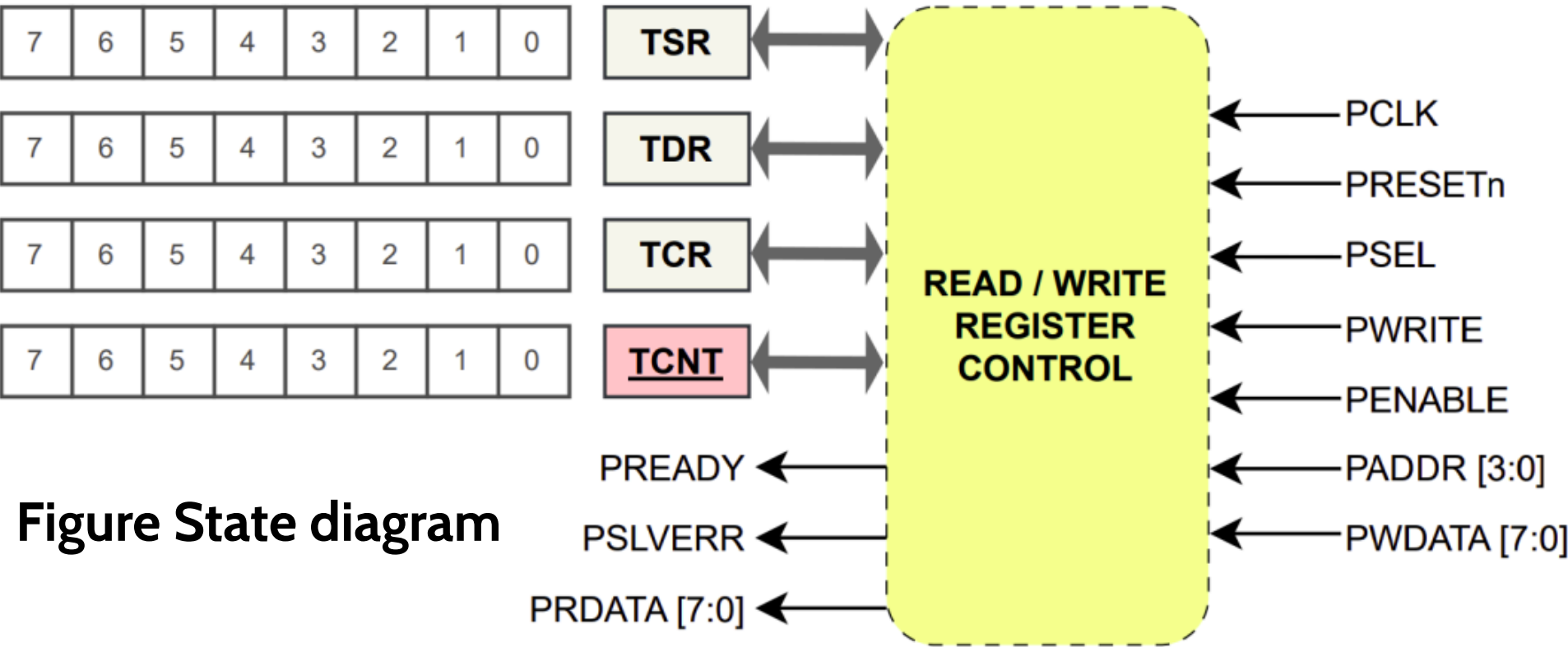


Figure State diagram

This module uses the AMBA 3 APB (Advanced Peripheral Bus) protocol to access internal control and status registers such as TCR, TCNT, TDR, and TSR.

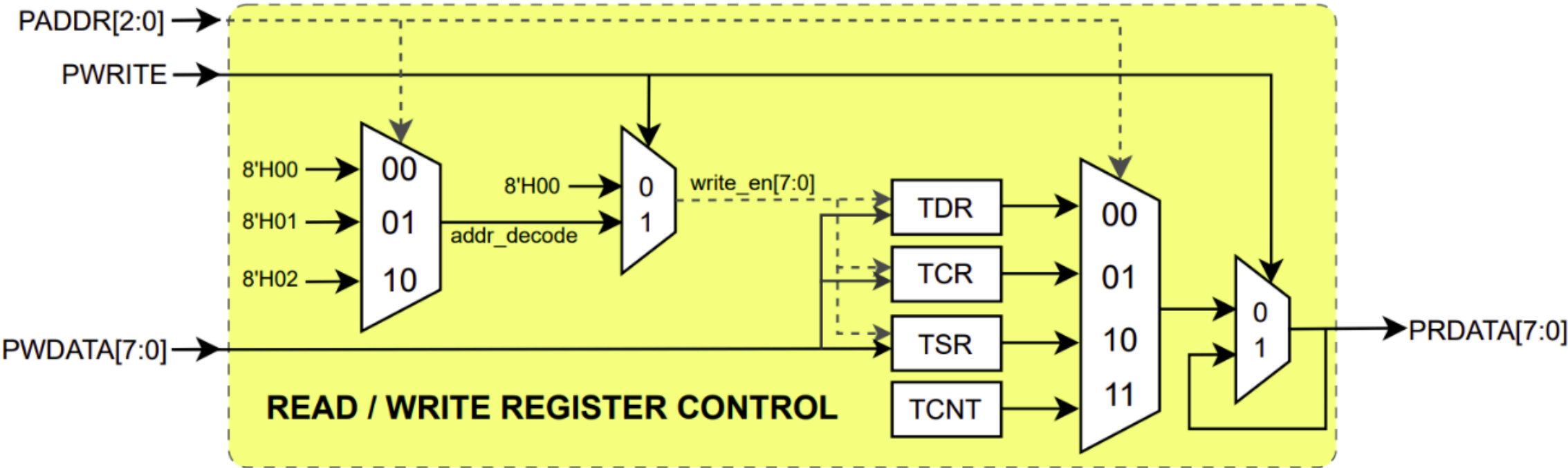


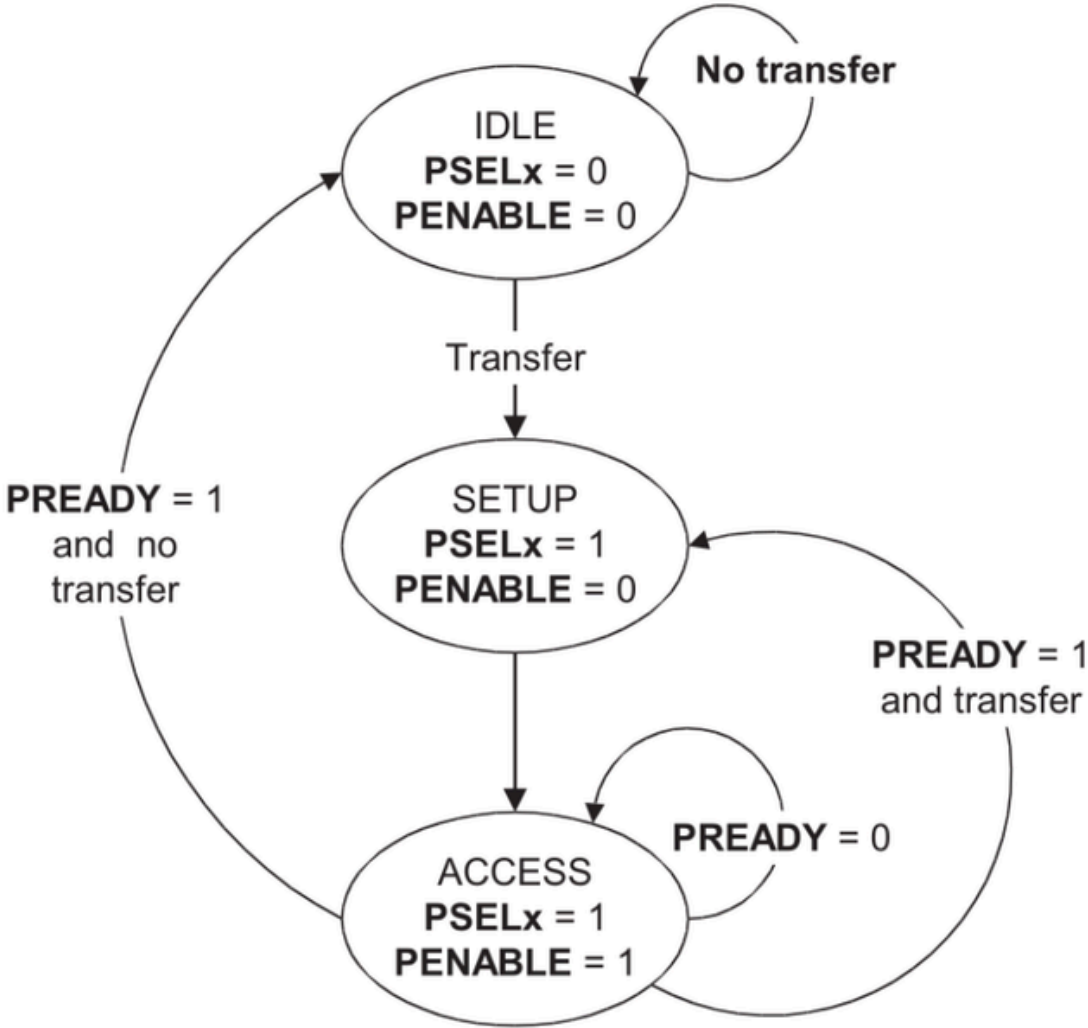
Figure READ/WRITE REGISTER CONTROL Circuit

Write path

- PADDR[2:0] → address decoder → write_en[3:0]
- PWDATA[7:0] is written into TDR/TCR/TSR when
- PSEL & PENABLE & PWRITE are HIGH.
- Write to TCNT is disabled (read-only counter).

Read path

- PADDR[2:0] selects one of TDR/TCR/TSR/TCNT through read mux.
- Selected register value is driven to PRDATA[7:0].



State machine control for APB protocol

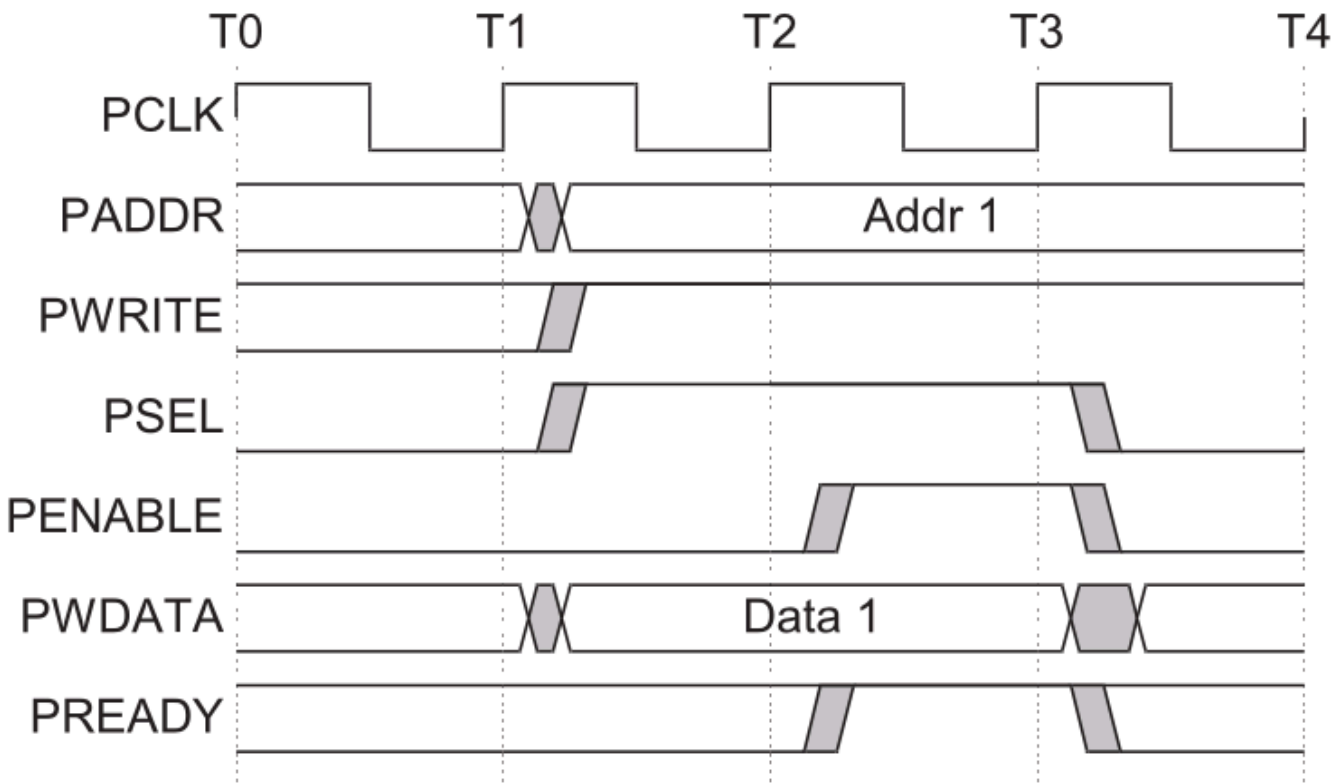


Figure Timing chart - Write transfer with no wait states

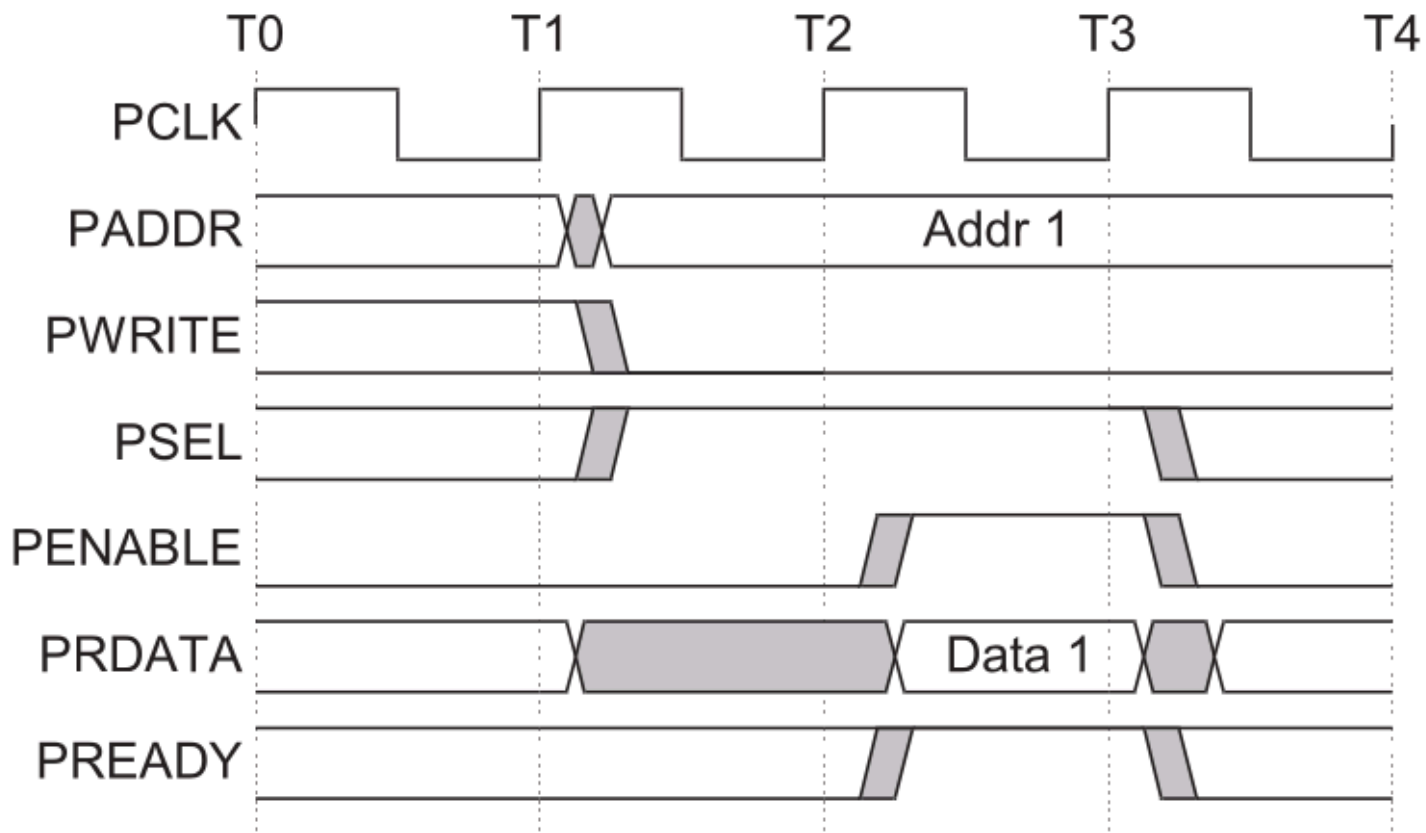


Figure Timing chart - Read transfer with no wait states


```

9  module rw_reg_control #(
10     parameter int ADDR_WIDTH = `ADDR_WIDTH
11 ) (
12     // APB INTERFACE SIGNALS
13     input wire          PCLK,
14     input wire          PRESETn,
15     input wire          PSEL,
16     input wire          PENABLE,
17     input wire          PWRITE,
18     input wire [ADDR_WIDTH-1:0] PADDR,
19     input wire [`DATA_WIDTH-1:0] PWDATA,
20
21     output wire [`DATA_WIDTH-1:0] PRDATA,
22     output wire          PREADY,
23     output wire          PSLVERR,
24
25     input wire          TMR_OVF,
26     input wire          TMR_UDF,
27
28     // INTERNAL REGISTER VALUES (to be used by other modules)
29     input wire [`DATA_WIDTH-1:0] TCNT,
30     output wire [`DATA_WIDTH-1:0] TDR,
31     output wire [`DATA_WIDTH-1:0] TCR,
32     output wire [`DATA_WIDTH-1:0] TSR
33 );
34
35     wire [`DATA_WIDTH-1:0] reg_TDR;
36     wire [`DATA_WIDTH-1:0] reg_TCR;
37     wire [`DATA_WIDTH-1:0] reg_TSR;
38
39     // Internal wires for connecting sub-modules
40     wire          apb_flag;
41     wire          apb_pready;
42     wire          apb_pslverr;
43     wire [`DATA_WIDTH-1:0] read_prdata; // Internal wire to connect read logic output to top-level PRDATA
44
49     // APB Transaction FSM
50     // This module manages the APB handshake and generates pready/pslverr signals.
51     APB_trans #(
52         .ADDR_WIDTH(ADDR_WIDTH)
53     ) u_apb_trans (
54         // input
55         .pclk      (PCLK),
56         .preset_n  (PRESETn),
57         .psel      (PSEL),
58         .penable   (PENABLE),
59         .pwrite    (PWRITE),
60         .paddr     (PADDR),
61         // output
62         .flag      (apb_flag),
63         .pready    (apb_pready),
64         .pslverr   (apb_pslverr)
65     );

```



```
67 // Write Logic Module
68 // This module handles writing to TDR, TCR, and TSR registers.
69 rw_write_logic #(
70     .ADDR_WIDTH(ADDR_WIDTH)
71 ) u_rw_write_logic (
72     // input
73     .pclk      (PCLK),
74     .preset_n  (PRESETn),
75     .flag      (apb_flag),
76     .pwrite    (PWRITE),
77     .paddr     (PADDR),
78     .pwrdata   (PWRDATA),
79     .ovf_flag  (TMR_OVF),
80     .udf_flag  (TMR_UDF),
81     // output
82     .TDR       (reg_TDR),
83     .TCR       (reg_TCR),
84     .TSR       (reg_TSR)
85 );
86
87 // Read Logic Module
88 // This module handles reading from all registers (TDR, TCR, TSR, TCNT).
89 rw_read_logic #(
90     .ADDR_WIDTH(ADDR_WIDTH)
91 ) u_rw_read_logic (
92     // input
93     .pclk      (PCLK),
94     .preset_n  (PRESETn),
95     .flag      (apb_flag),
96     .pwrite    (PWRITE),
97     .paddr     (PADDR),
98     .TDR       (reg_TDR),
99     .TCR       (reg_TCR),
100    .TSR       (reg_TSR),
101    .TCNT      (TCNT),
102    // output
103    .prdata    (read_prdata)
104 );
---
```

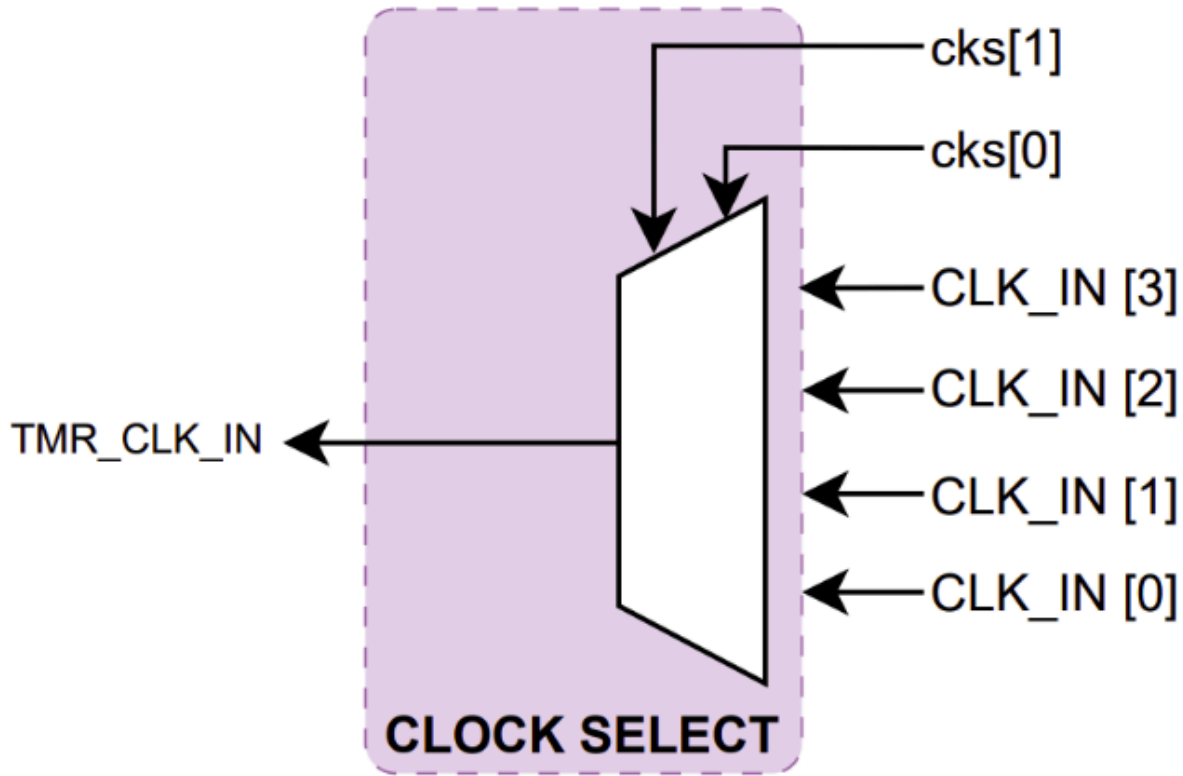


Figure CLOCK SELECT Block diagram

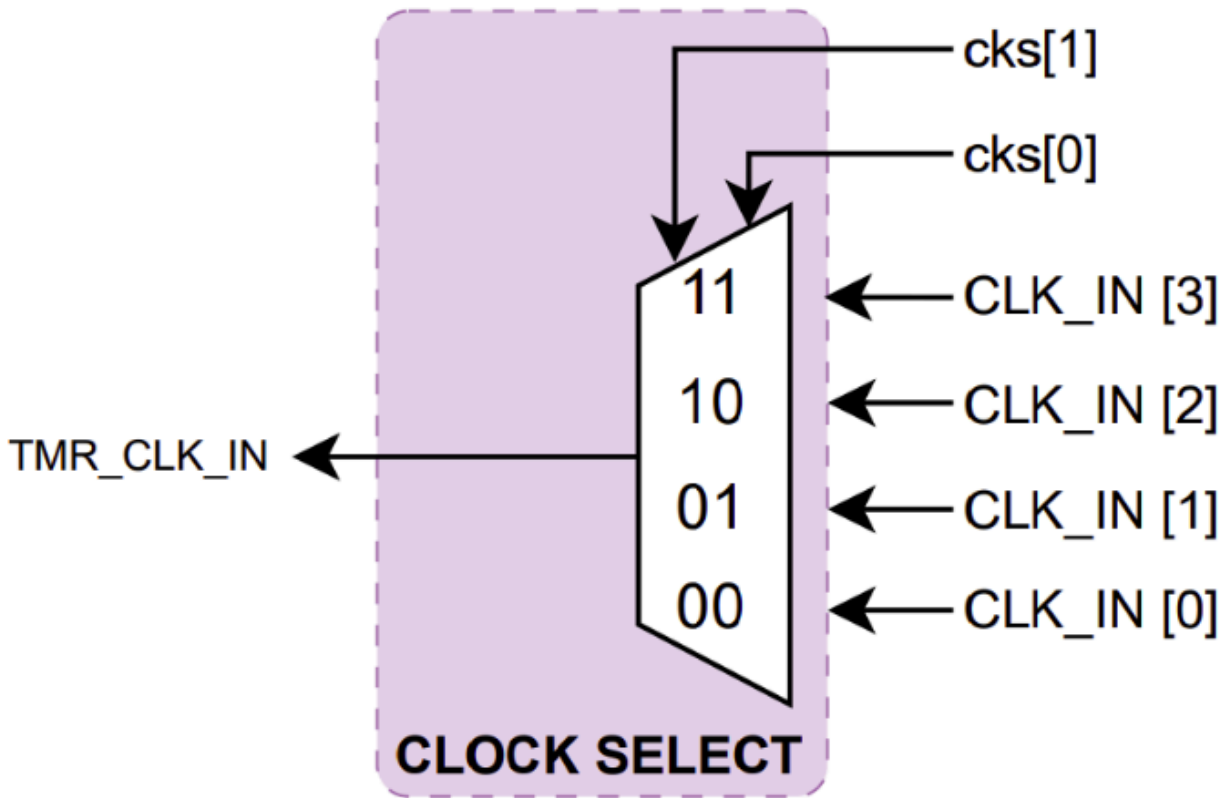


Figure CLOCK SELECT Circuit

This block selects one clock signal among multiple inputs (CLK_IN[3:0]) based on a 2-bit selector (TCR[1:0]).

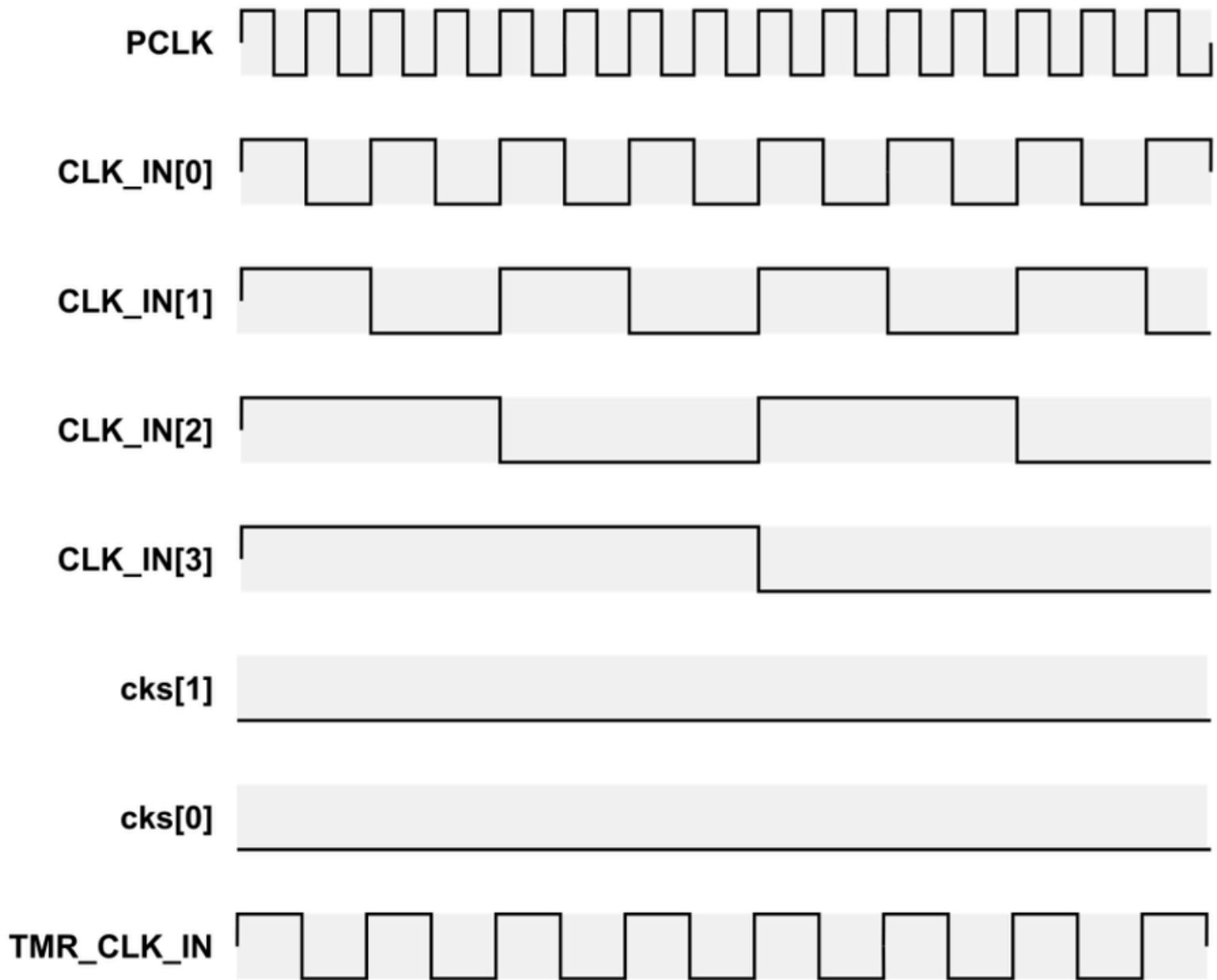


Figure CLOCK SELECT Timing chart

```
75 // -----
76 // Module: clk_sel_synced_alt
77 // Description:
78 //   - Registers the clock select input (cks).
79 //   - Selects one of the 4 divided clocks using the registered cks value.
80 //   - Registers the selected clock output with asynchronous reset.
81 // -----
82 module clk_sel_synced_alt (
83     input wire      pclk,
84     input wire      preset_n,
85     input wire [3:0] CLK_IN,
86     input wire [1:0] cks, // clk_select signal
87     output reg      TMR_CLK_IN
88 );
89 // Internal signals for synchronization and edge detection
90 reg [1:0] cks_r;
91 reg      TMR_CLK_IN_mux_out;
92
93 // Register the clock select signal
94 // This is a synchronous reset to ensure consistent behavior across FFs.
95 always @(posedge pclk or negedge preset_n) begin
96     if (!preset_n) cks_r <= 2'b0;
97     else          cks_r <= cks;
98 end
99
100 // Clock Select MUX (Combinational Logic)
101 always @(*) begin
102     case (cks_r)
103         2'b00: TMR_CLK_IN_mux_out = CLK_IN[0];
104         2'b01: TMR_CLK_IN_mux_out = CLK_IN[1];
105         2'b10: TMR_CLK_IN_mux_out = CLK_IN[2];
106         2'b11: TMR_CLK_IN_mux_out = CLK_IN[3];
107         default: TMR_CLK_IN_mux_out = 1'b0;
108     endcase
109 end
110
111 // Final output register with asynchronous reset
112 // This ensures TMR_CLK_IN is a registered output and is cleared immediately on reset.
113 always @(posedge pclk or negedge preset_n) begin
114     if (!preset_n) TMR_CLK_IN <= 1'b0;
115     else          TMR_CLK_IN <= TMR_CLK_IN_mux_out;
116 end
117 endmodule
```


Logic Control Block manages core timer operations based on TCR and TDR inputs.

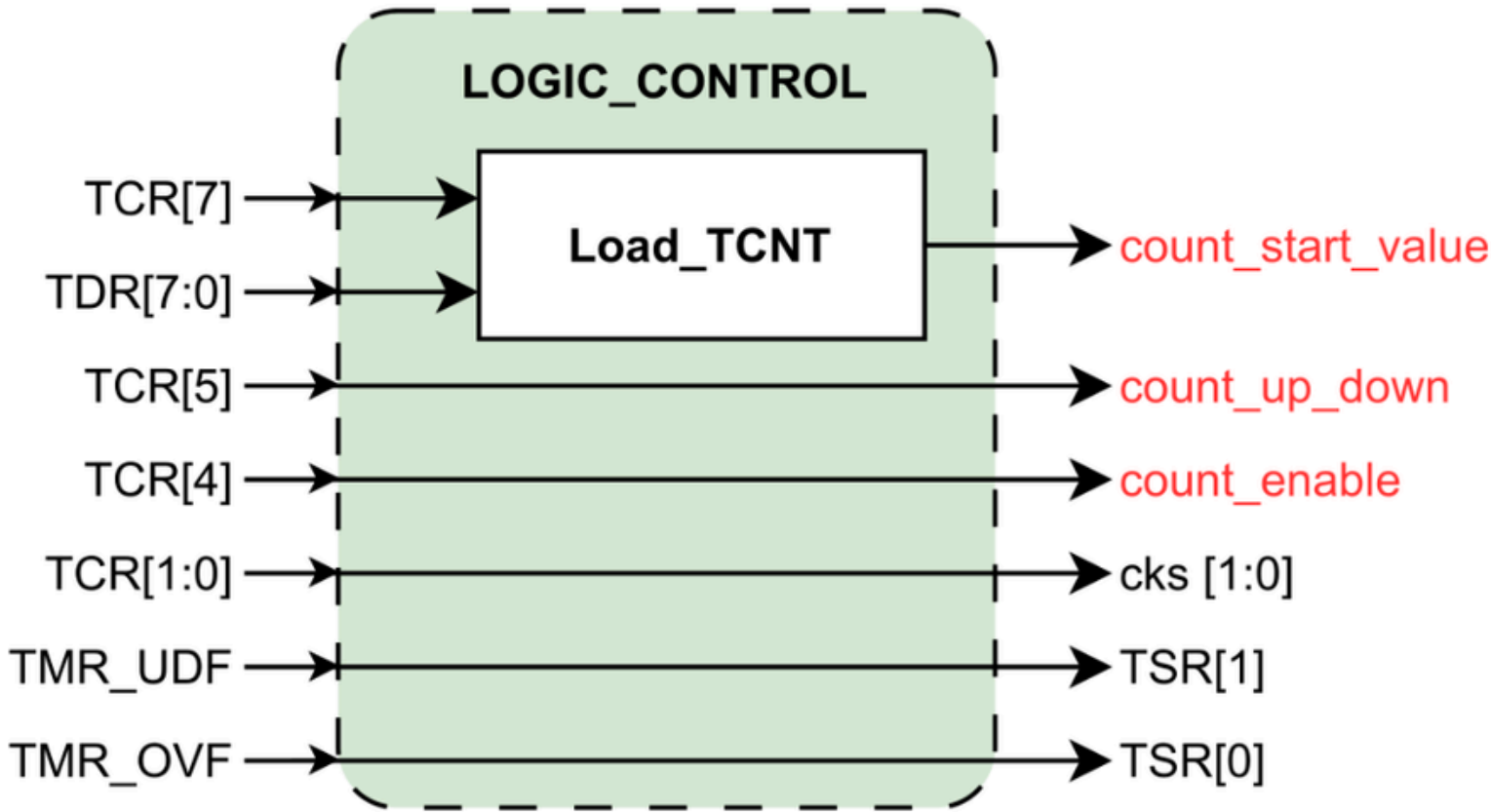


Figure CONTROL LOGIC Block diagram

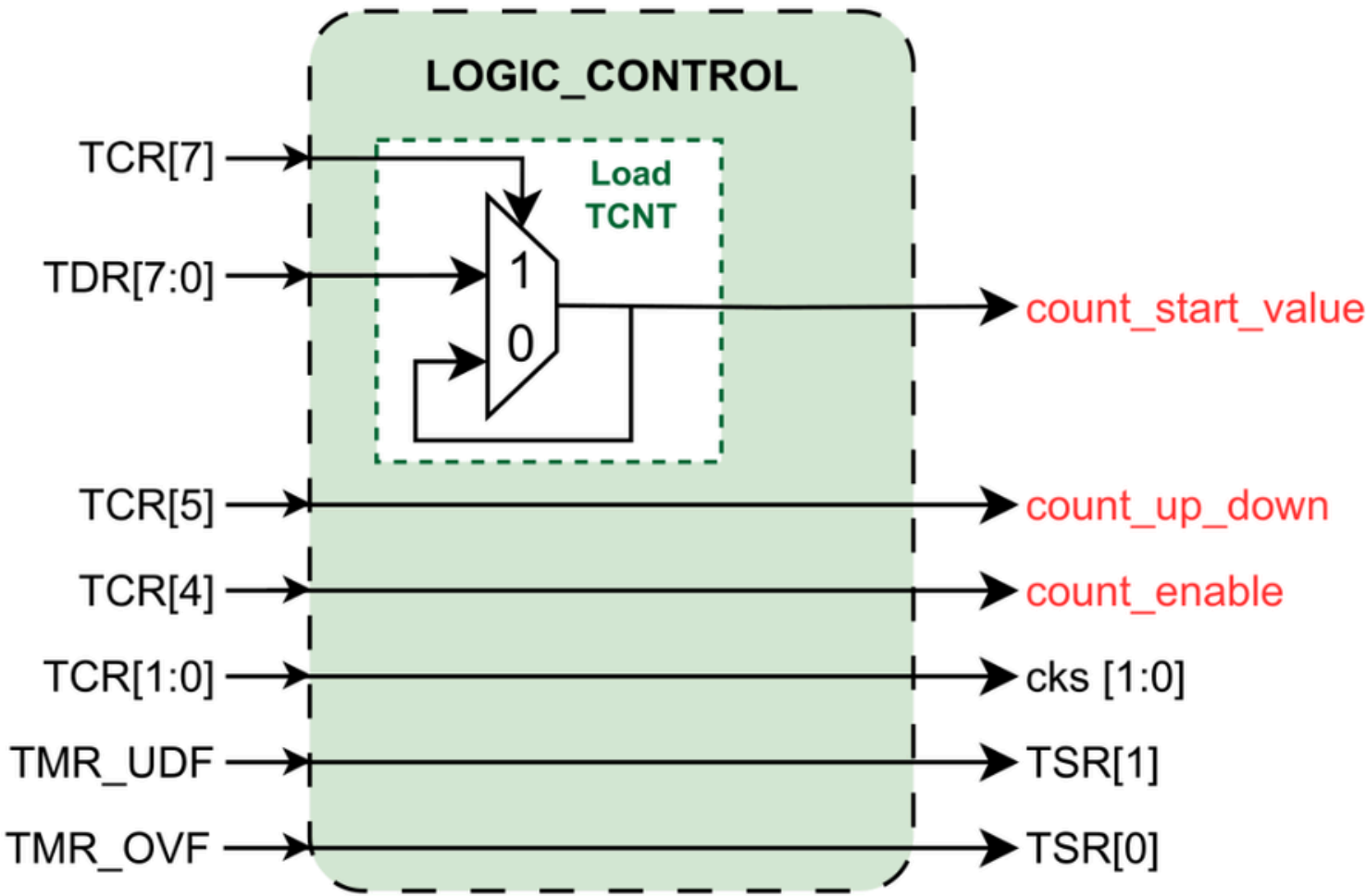


Figure LOGIC CONTROL circuit

```
31 module logic_control (  
32     input wire [`DATA_WIDTH-1:0] TDR,           // Timer Data Register  
33     input wire [`DATA_WIDTH-1:0] TCR,           // Timer Control Register  
34     output wire [`DATA_WIDTH-1:0] count_start_value, // Value to load into TCNT  
35     output wire count_load,                     // Load control  
36     output wire count_enable,                   // Counter enable  
37     output wire count_up_down,                 // Direction control  
38     output wire [1:0] cks                      // Clock source select  
39 );  
40  
41 // Load value to TCNT if TCR[7] is set, otherwise 0  
42 assign count_start_value = (TCR[`TCR_LOAD_BIT]) ? TDR : {'DATA_WIDTH{1'b0}};  
43  
44 // Extract control signals  
45 assign count_load = TCR[`TCR_LOAD_BIT];  
46 assign count_up_down = TCR[`TCR_UPDOWN_BIT];  
47 assign count_enable = TCR[`TCR_EN_BIT];  
48 assign cks = {TCR[`TCR_CKS_MSB], TCR[`TCR_CKS_LSB]};  
49  
50 endmodule
```

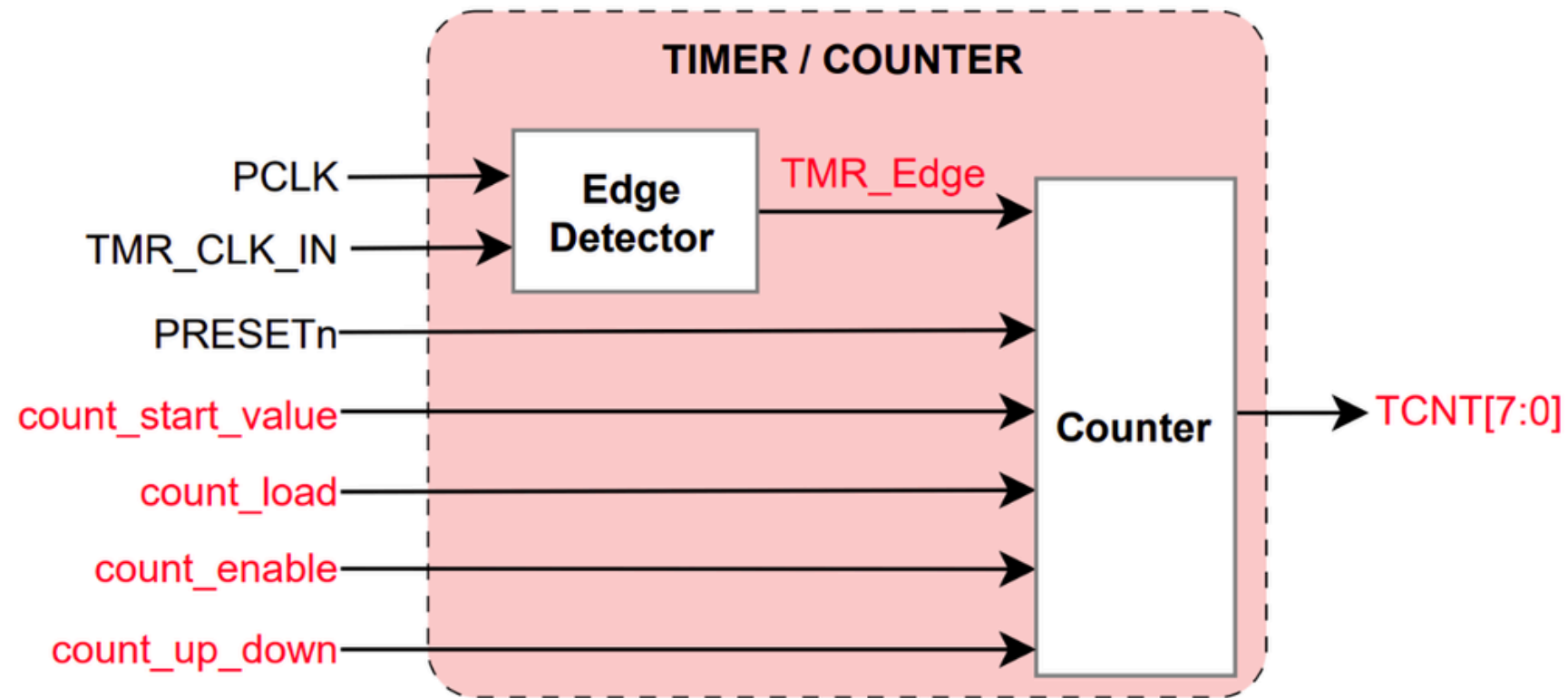


Figure COUNTER UNIT Block diagram

Functional overview:

- TCNT resets to 0 when PRESETn = 0.
- If count_enable = 1, counting begins.
- When count_load = 1, the counter loads count_start_value from TDR.
- Output TCNT[7:0] reflects the current counter value.
- Overflow and underflow events are detected and trigger TMR_OVF or TMR_UDF flags.
- These flags are stored in the Timer Status Register (TSR) for software access.

The COUNTER UNIT unit performs up/down counting based on control inputs from the Control Logic. Operates synchronously with the TMR_Edge signal (generated from the selected clock via the Clock Select block).

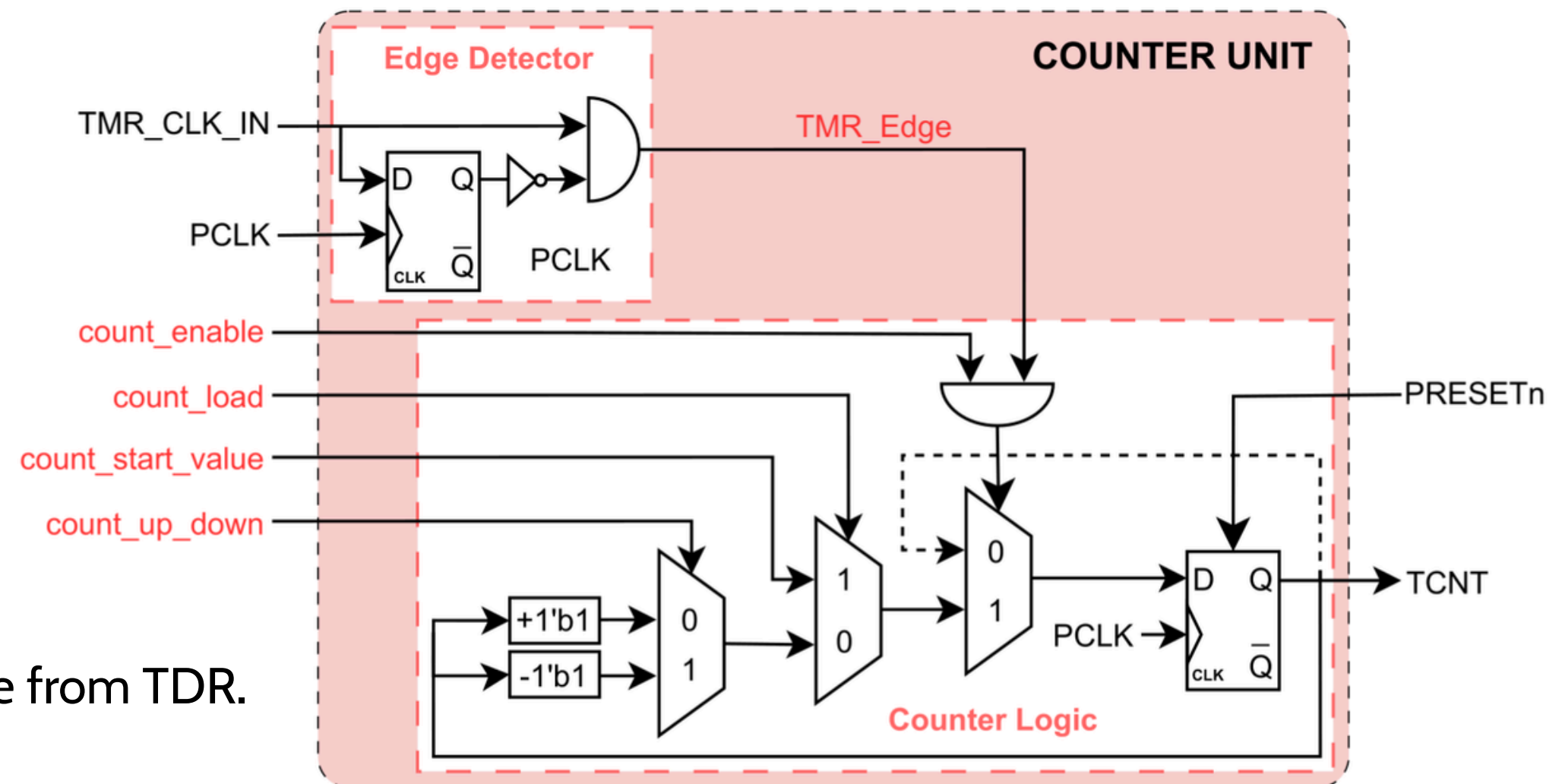


Figure COUNTER UNIT circuit

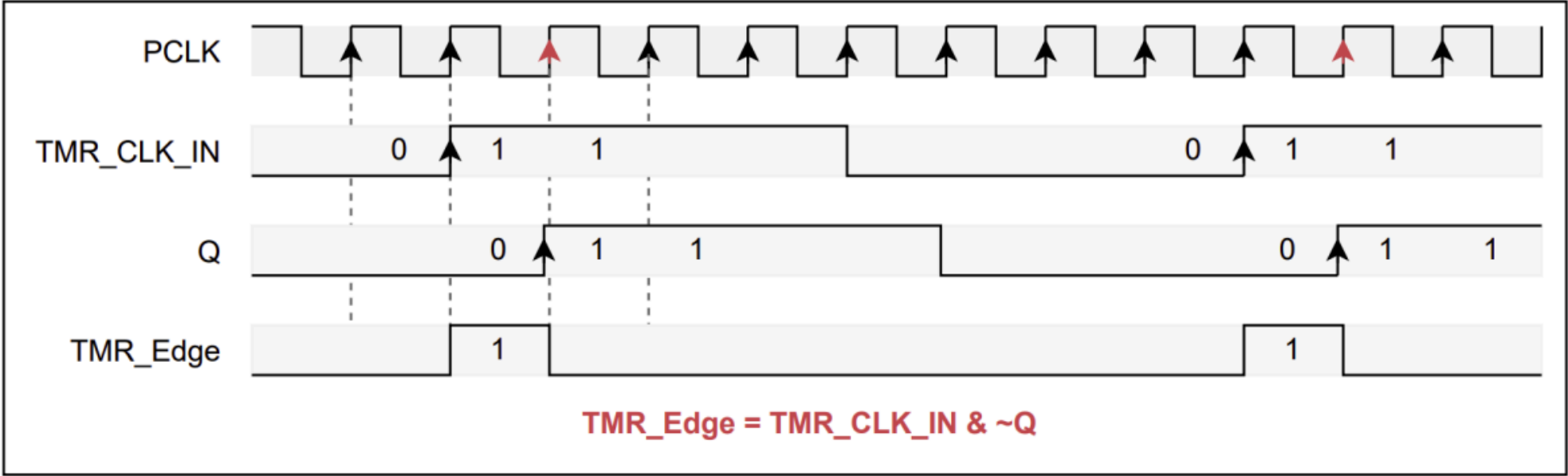


Figure Detect Rising Edge Timing chart

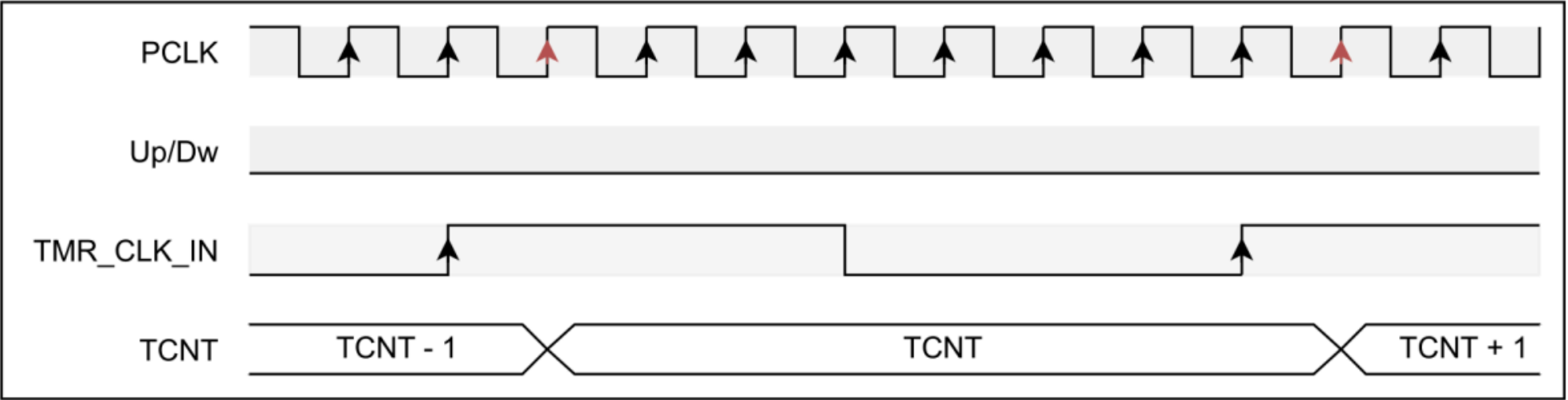


Figure TCNT Count up Timing chart (Rising Edge)

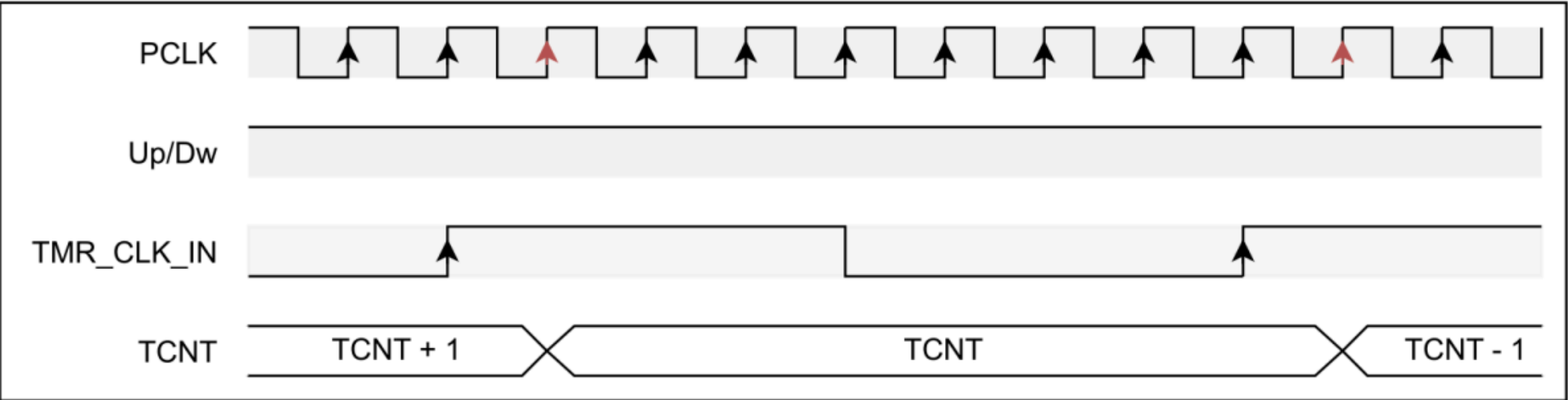


Figure TCNT Count down Timing chart (Rising Edge)


```

49 // -----
50 // Module: pos_edge_detect_synced
51 // Description:
52 //   - Performs a one-stage synchronization of the selected clock to pclk.
53 //   - Detects the rising edge and generates a one-cycle pulse.
54 // -----
55 module pos_edge_detect_synced (
56     input wire pclk,
57     input wire preset_n,
58     input wire TMR_CLK_IN,
59     output wire TMR_Edge
60 );
61 // Delayed version of TMR_CLK_IN using DFF
62 reg TMR_CLK_IN_d;
63
64 // Asynchronous reset for the synchronizer FF
65 always @(posedge pclk or negedge preset_n) begin
66     if (!preset_n) TMR_CLK_IN_d <= 1'b0; // Reset delayed signal
67     else TMR_CLK_IN_d <= TMR_CLK_IN; // Capture previous state
68 end
69
70 // Generate a one-clock-cycle pulse on the rising edge
71 assign TMR_Edge = TMR_CLK_IN & (~TMR_CLK_IN_d);
72
73 endmodule

```

```

35 module cnt_unit (
36     input wire pclk, // System clock
37     input wire preset_n, // Active-low synchronous reset
38
39     input wire TMR_Edge, // Clock edge from selected clock source
40     input wire [`DATA_WIDTH-1:0] count_start_value, // Load value when count_load is asserted
41     input wire count_load, // Load enable
42     input wire count_enable, // Enable counting
43     input wire count_up_down, // 0 = up count, 1 = down count
44
45     output reg [`DATA_WIDTH-1:0] TCNT // Current count value
46 );
47
48 always @(posedge pclk or negedge preset_n) begin
49     if (!preset_n) begin
50         TCNT <= `TCNT_RST; // Reset counter to default value
51     end else begin
52         if (count_load) begin
53             TCNT <= count_start_value;
54         end else if (count_enable && TMR_Edge) begin
55             if (!count_up_down) begin
56                 // Up count: wrap to 0 when max
57                 TCNT <= (TCNT == [`DATA_WIDTH{1'b1}]) ? [`DATA_WIDTH{1'b0}] : TCNT + 1'b1;
58             end else begin
59                 // Down count: wrap to max when 0
60                 TCNT <= (TCNT == [`DATA_WIDTH{1'b0}]) ? [`DATA_WIDTH{1'b1}] : TCNT - 1'b1;
61             end
62         end else begin
63             // Hold current value
64             TCNT <= TCNT;
65         end
66     end
67 end
68
69 endmodule

```

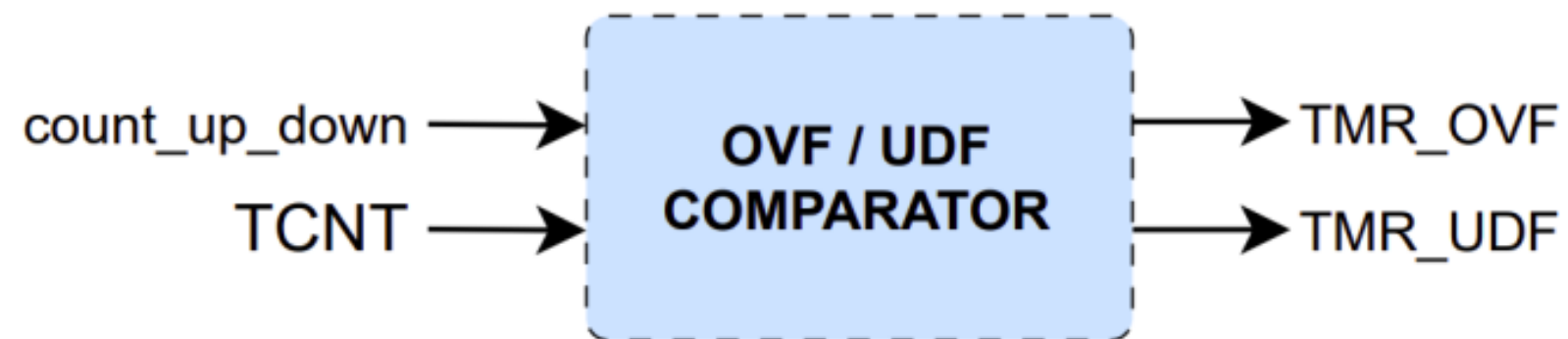


Figure OVERFLOW/UNDERFLOW COMPARATOR Block diagram

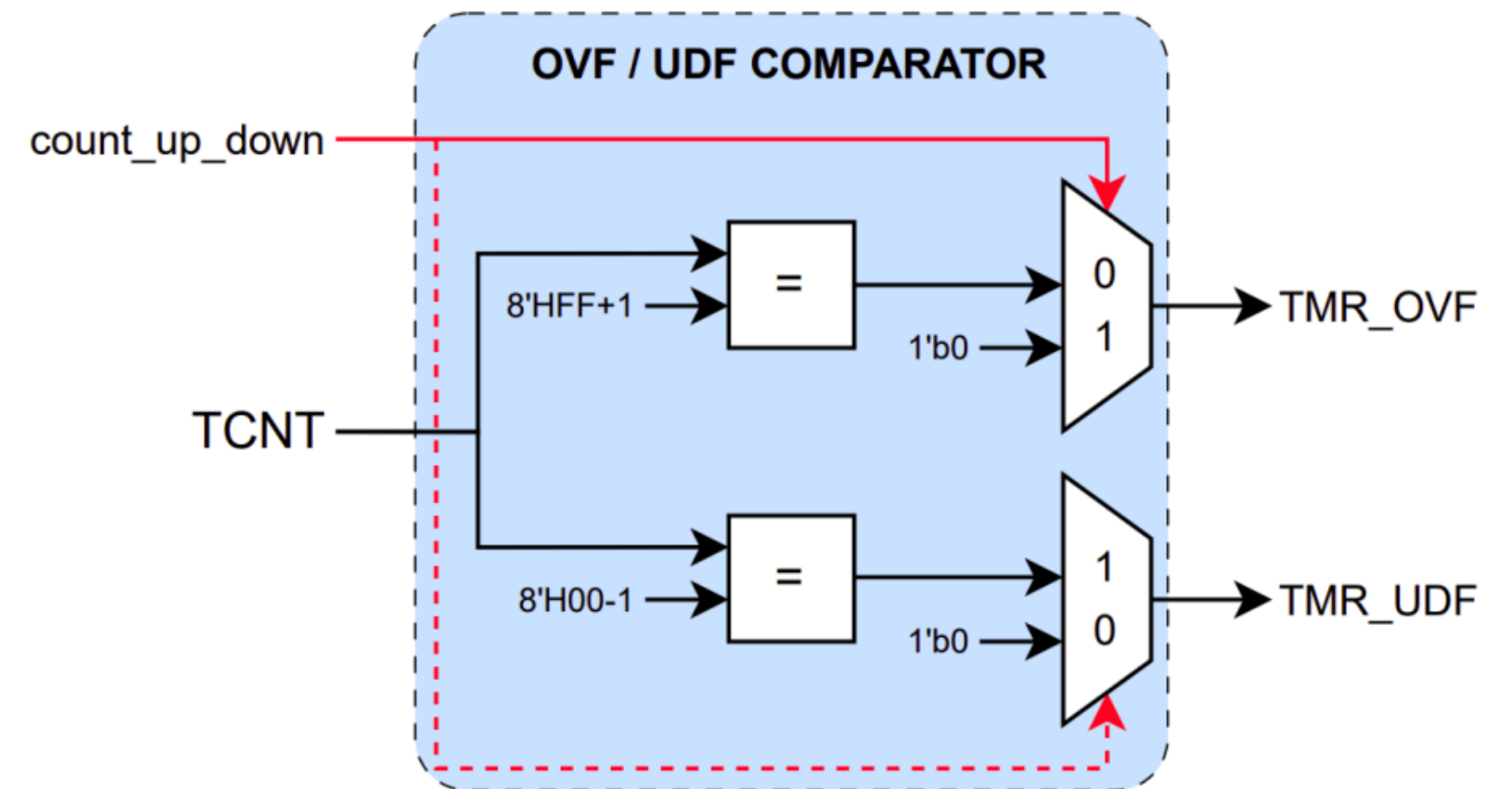


Figure OVERFLOW/UNDERFLOW COMPARATOR circuit

This block monitors the counter value (TCNT) to detect overflow and underflow events:

- Count-up mode ($\text{count_up_down} = 0$): When TCNT reaches $8'hFF+1$, the TMR_OVF flag is set high for one clock cycle to indicate an overflow.
- Count-down mode ($\text{count_up_down} = 1$): When TCNT reaches $8'h00+1$, the TMR_UDF flag is set high for one clock cycle to indicate an underflow.

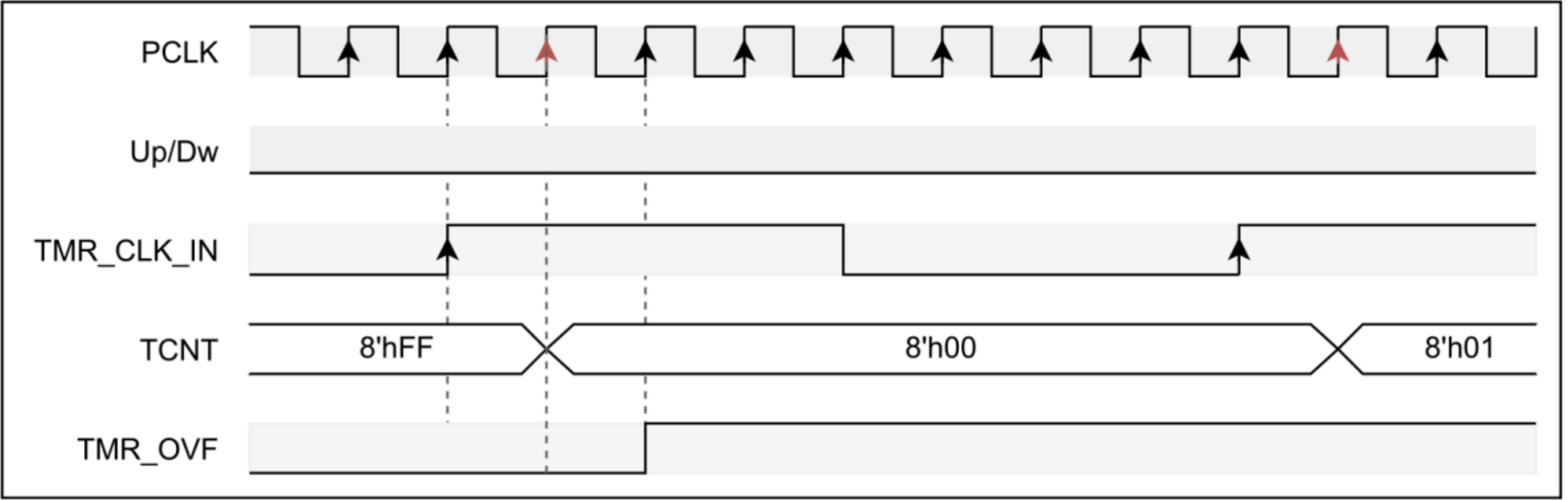


Figure Timing of Overflow Flag (TMR_OVF) Setting (Rising Edge)

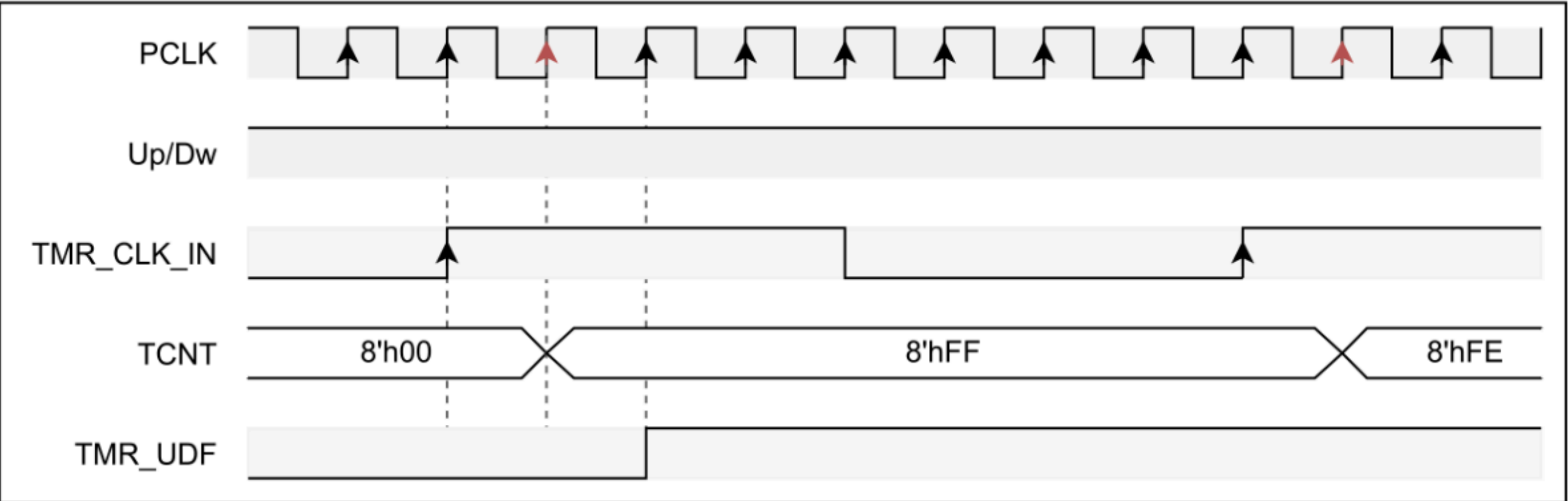


Figure Timing of Underflow Flag (TMR_UDF) Setting (Rising Edge)


```

33 module ovf_udf_comp (
34     input wire          pclk,          // System clock
35     input wire          preset_n,      // Active-low synchronous reset
36
37     input wire [`DATA_WIDTH-1:0] TCNT, // Current counter value
38     input wire          count_enable,  // Count enable signal
39     input wire          count_up_down, // 0: count up, 1: count down
40
41     input wire [`DATA_WIDTH-1:0] TSR,  // Timer Control Register
42     output reg          TMR_OVF,       // Overflow flag
43     output reg          TMR_UDF       // Underflow flag
44 );
45
46 reg [`DATA_WIDTH-1:0] TCNT_d; // previous TCNT
47 reg [`DATA_WIDTH-1:0] TSR_d;  // previous TSR
48

```

```

49 always @(posedge pclk or negedge preset_n) begin
50     if (!preset_n) begin
51         // Reset all flags
52         TMR_OVF <= 1'b0;
53         TMR_UDF <= 1'b0;
54         TCNT_d <= {'DATA_WIDTH{1'b0}};
55         TSR_d <= {'DATA_WIDTH{1'b0}};
56     end else begin
57         // detect SW-clear
58         if ((TMR_OVF && !TSR[0] && TSR_d[0]) || (TMR_UDF && !TSR[1] && TSR_d[1])) begin
59             TMR_OVF <= (TMR_OVF && !TSR[0] && TSR_d[0]) ? 1'b0 : TMR_OVF;
60             TMR_UDF <= (TMR_UDF && !TSR[1] && TSR_d[1]) ? 1'b0 : TMR_UDF;
61         end else if (count_enable) begin
62             // Detect overflow/underflow only if enabled
63             // Check counting direction
64             if (!count_up_down) begin
65                 // clear UDF trong mode up
66                 TMR_UDF <= 1'b0;
67                 // Counting up: TCNT == 8'hFF means the counter is about to overflow.
68                 // The real overflow would happen on the next count (TCNT wraps to 8'h00).
69                 if (TCNT_d == {'DATA_WIDTH{1'b1}} && TCNT == {'DATA_WIDTH{1'b0}}) TMR_OVF <= 1'b1;
70                 else TMR_OVF <= TMR_OVF;
71             end else begin
72                 // clear OVF trong mode down
73                 TMR_OVF <= 1'b0;
74                 // Counting down: TCNT == 8'h00 means the counter is about to underflow.
75                 // The real underflow would happen on the next count (TCNT wraps to 8'hFF).
76                 if (TCNT_d == {'DATA_WIDTH{1'b0}} && TCNT == {'DATA_WIDTH{1'b1}}) TMR_UDF <= 1'b1;
77                 else TMR_UDF <= TMR_UDF;
78             end
79         end else begin
80             // No counting → hold them
81             TMR_OVF <= TMR_OVF;
82             TMR_UDF <= TMR_UDF;
83         end
84
85         // Save current value
86         TCNT_d <= TCNT;
87         // Save current status
88         TSR_d <= TSR;
89     end
90 end
91
92
93 endmodule

```

- The APB bus of the timer is encapsulated in a SystemVerilog interface named **apb_if**.
- **apb_if** groups **clock**, **reset** and **all APB transfer signals** into a **single port**.
- This simplifies the connection between **VIP (APB master)** and **Timer IP (APB slave)** and clearly separates them using modports.

Clock / Reset signals

Signal	Direction	Width	Description
PCLK	Input	1	APB bus clock
PRESETn	Input	1	Active-low reset for APB logic

APB transfer signals

Signal	Direction	Width	Description
PSEL	Input	1	Selects this APB slave
PENABLE	Input	1	Indicates the enable phase of a transfer
PWRITE	Input	1	1: write transfer, 0: read transfer
PADDR	Input	8	Register address (TCR, TSR, TDR, ...)
PWDATA	Input	8	Write data bus
PRDATA	Output	8	Read data bus
PREADY	Output	1	Transfer ready / wait-state control
PSLVERR	Output	1	Error response (e.g. invalid address access)

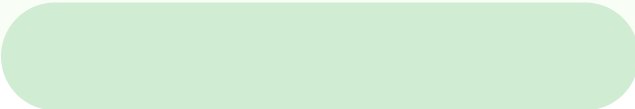

```
MAKEFILE_COVERAGE > rtl > apb_if.sv
1  `ifndef APB_IF_SV
2  `define APB_IF_SV
3
4  `include "reg_def.sv"
5
6  // -----
7  // Simple APB interface
8  // -----
9  interface apb_if #(
10     parameter int ADDR_WIDTH = `ADDR_WIDTH,
11     parameter int DATA_WIDTH = `DATA_WIDTH
12 )()
13     input  logic PCLK,
14     input  logic PRESETn
15 );
16
17     // APB signals
18     logic          PSEL;
19     logic          PENABLE;
20     logic          PWRITE;
21     logic [ADDR_WIDTH-1:0] PADDR;
22     logic [DATA_WIDTH-1:0] PWDATA;
23     logic [DATA_WIDTH-1:0] PRDATA;
24     logic          PREADY;
25     logic          PSLVERR;
26
27     // Master view (CPU / driver)
28     modport MASTER (
29         input  PCLK, PRESETn,
30         input  PREADY, PRDATA, PSLVERR,
31         output PSEL, PENABLE, PWRITE, PADDR, PWDATA
32     );
33
34     // Slave view (DUT)
35     modport SLAVE (
36         input  PCLK, PRESETn,
37         input  PSEL, PENABLE, PWRITE, PADDR, PWDATA,
38         output PREADY, PRDATA, PSLVERR
39     );
40
41 endinterface
42
43 `endif // APB_IF_SV
```


2.3. Top-level RTL

```
21 module ip_TIMER #(
22     parameter int ADDR_WIDTH = `ADDR_WIDTH
23 );
24     input wire [3:0] CLK_IN,
25     apb_if.SLAVE      apb,
26     output wire       TMR_OVF,
27     output wire       TMR_UDF
28 );
29
30 // =====
31 // Internal wires/regs
32 // =====
33
34 // Register values (TDR, TCR, TSR, TCNT)
35 wire [`DATA_WIDTH-1:0] TDR_reg;
36 wire [`DATA_WIDTH-1:0] TCR_reg;
37 wire [`DATA_WIDTH-1:0] TSR_reg;
38 wire [`DATA_WIDTH-1:0] TCNT_reg;
39
40 // Control signals
41 wire [`DATA_WIDTH-1:0] count_start_value;
42 wire                   count_load;
43 wire                   count_enable;
44 wire                   count_up_down;
45 wire [1:0]             cks;
46
47 // Clock edge from selected clock source
48 wire                   TMR_Edge;
49
50 // Internal OVF/UDF flags from comparator
51 wire                   TMR_OVF_int;
52 wire                   TMR_UDF_int;
53
```

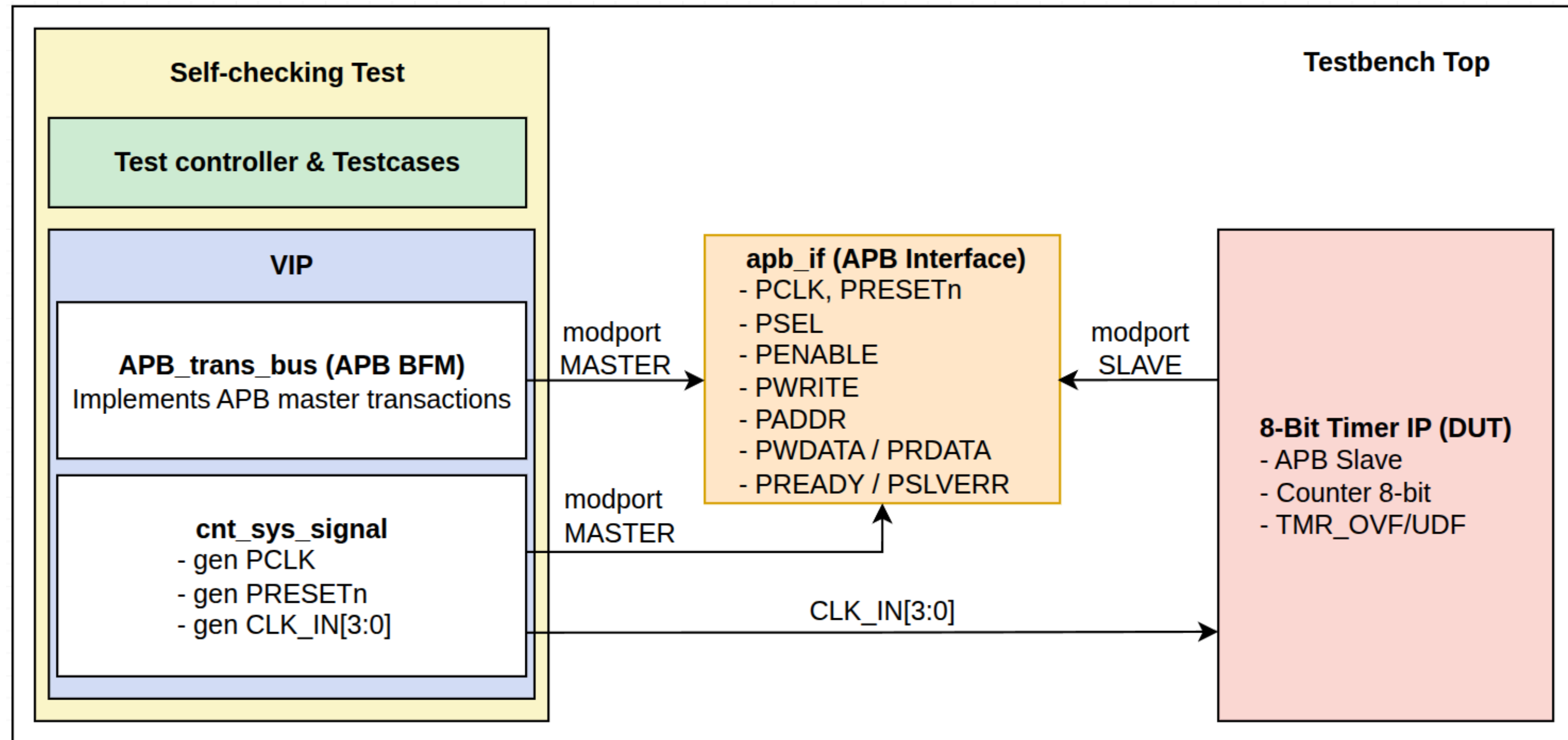
```
54 // =====
55 // 1. APB READ/WRITE REGISTER CONTROLLER
56 // =====
57 rw_reg_control #(
58     .ADDR_WIDTH(ADDR_WIDTH)
59 ) u_rw_reg_control (
60     .PCLK      (apb.PCLK),
61     .PRESETn    (apb.PRESETn),
62     .PSEL       (apb.PSEL),
63     .PENABLE     (apb.PENABLE),
64     .PWRITE      (apb.PWRITE),
65     .PADDR       (apb.PADDR),
66     .PWDATA      (apb.PWDATA),
67     .PRDATA      (apb.PRDATA),
68     .PREADY      (apb.PREADY),
69     .PSLVERR     (apb.PSLVERR),
70
71     .TMR_OVF     (TMR_OVF_int),
72     .TMR_UDF     (TMR_UDF_int),
73
74     .TCNT        (TCNT_reg),
75     .TDR         (TDR_reg),
76     .TCR         (TCR_reg),
77     .TSR         (TSR_reg)
78 );
79
80 // =====
81 // 2. CLOCK SELECT + EDGE DETECTION
82 // =====
83 pos_cnt_edge_detect u_pos_cnt_edge_detect (
84     .pclk      (apb.PCLK),
85     .preset_n  (apb.PRESETn),
86     .CLK_IN    (CLK_IN),
87     .cks       (cks),
88     .TMR_Edge  (TMR_Edge)
89 );
90
```

```
91 // =====
92 // 3. LOGIC CONTROL
93 // =====
94 logic_control u_logic_control (
95     .TDR          (TDR_reg),
96     .TCR          (TCR_reg),
97     .count_start_value (count_start_value),
98     .count_load      (count_load),
99     .count_enable     (count_enable),
100     .count_up_down    (count_up_down),
101     .cks             (cks)
102 );
103
104 // =====
105 // 4. COUNTER UNIT
106 // =====
107 cnt_unit u_cnt_unit (
108     .pclk          (apb.PCLK),
109     .preset_n       (apb.PRESETn),
110     .TMR_Edge       (TMR_Edge),
111     .count_start_value (count_start_value),
112     .count_load      (count_load),
113     .count_enable     (count_enable),
114     .count_up_down    (count_up_down),
115     .TCNT           (TCNT_reg)
116 );
117
118 // =====
119 // 5. OVERFLOW / UNDERFLOW COMPARATOR
120 // =====
121 ovf_udf_comp u_ovf_udf_comp (
122     .pclk          (apb.PCLK),
123     .preset_n       (apb.PRESETn),
124     .TCNT           (TCNT_reg),
125     .count_enable     (count_enable),
126     .count_up_down    (count_up_down),
127     .TSR            (TSR_reg),
128     .TMR_OVF         (TMR_OVF_int),
129     .TMR_UDF         (TMR_UDF_int)
130 );
```



3. Design Verification

- 3.1. Testbench Environment
- 3.2. VIP
- 3.3. Test plan
- 3.4. Testbench top



The 8-bit Timer IP is verified with a self-checking testbench.

A test controller triggers APB-based testcases (reset, configuration, overflow/underflow, pause/resume) and checks results.

The VIP consists of APB_trans_bus (APB master BFM) and cnt_sys_signal (generator for PCLK, PRESETn, CLK_IN[3:0]).

All APB signals are encapsulated in the apb_if interface, which connects the VIP to the Timer IP (DUT) via MASTER/SLAVE modports.

APB_trans_bus – APB BFM (master)

- Implements APB master transactions (read/write, configuration sequences, etc.).
- Drives PSEL, PENABLE, PWRITE, PADDR, PWDATA through the apb_if.MASTER modport.
- Provides high-level tasks so that testcases can focus on scenarios instead of signal toggling.

cnt_sys_signal – clock & reset generator

- Generates the APB bus clock PCLK, the active-low reset PRESETn, and the CLK_IN[3:0] clock sources for the timer.
- Ensures a consistent and controllable timing environment for all tests.

Reuse benefit

- This VIP can be reused to verify other APB-based slave IPs by simply connecting the same apb_if interface to a different DUT.

APB_trans_bus – APB BFM (master)

MAKEFILE_COVERAGE > tb > VIP >  APB_trans_bus.sv

```
7 // APB master BFM use apb_if.MASTER
8 module APB_trans_bus #(
9     parameter int ADDR_WIDTH = `ADDR_WIDTH,
10    parameter int DATA_WIDTH = `DATA_WIDTH
11 ) (
12     apb_if.MASTER apb
13 );
14
15 // -----
16 // INIT: set for IDLE
17 // -----
18 initial begin
19     apb.PSEL    = 1'b0;
20     apb.PENABLE = 1'b0;
21     apb.PWRITE  = 1'b0;
22     apb.PADDR   = '0;
23     apb.PWDATA  = '0;
24 end
```

APB_trans_bus – APB BFM (master)

```

26 // -----
27 // Low-level APB write (no wait-state)
28 // -----
29 task automatic apb_write (
30     input logic [ADDR_WIDTH-1:0] addr,
31     input logic [DATA_WIDTH-1:0] data
32 );
33     begin
34         // IDLE -> SETUP
35         @(posedge apb.PCLK);
36         apb.PADDR    <= addr;
37         apb.PWDATA    <= data;
38         apb.PWRITE    <= 1'b1;
39         apb.PSEL      <= 1'b1;
40         apb.PENABLE   <= 1'b0;
41
42         // SETUP -> ACCESS
43         @(posedge apb.PCLK);
44         apb.PENABLE   <= 1'b1;
45
46         // ACCESS: wait PREADY (if design always has PREADY=1 => only 1 cycle)
47         do @(posedge apb.PCLK); while (!apb.PREADY);
48
49         // Kết thúc: về IDLE
50         apb.PSEL      <= 1'b0;
51         apb.PENABLE   <= 1'b0;
52         apb.PWRITE    <= 1'b0;
53         apb.PADDR     <= '0;
54         apb.PWDATA    <= '0;
55     end
56 endtask

```

```

58 // -----
59 // Low-level APB read
60 // -----
61 task automatic apb_read (
62     input logic [ADDR_WIDTH-1:0] addr,
63     output logic [DATA_WIDTH-1:0] data
64 );
65     begin
66         // IDLE -> SETUP
67         @(posedge apb.PCLK);
68         apb.PADDR    <= addr;
69         apb.PWRITE    <= 1'b0;
70         apb.PSEL      <= 1'b1;
71         apb.PENABLE   <= 1'b0;
72
73         // SETUP -> ACCESS
74         @(posedge apb.PCLK);
75         apb.PENABLE   <= 1'b1;
76
77         // ACCESS: wait PREADY, get PRDATA
78         do @(posedge apb.PCLK); while (!apb.PREADY);
79         data = apb.PRDATA;
80
81         // Kết thúc: về IDLE
82         apb.PSEL      <= 1'b0;
83         apb.PENABLE   <= 1'b0;
84         apb.PADDR     <= '0;
85     end
86 endtask

```


cnt_sys_signal – clock & reset generator

MAKEFILE_COVERAGE > tb > VIP > cnt_sys_signal.sv

```

4 // =====
5 // Module: sys_signal
6 // Description: Generates the main system clock and active-low reset signal.
7 // Parameters:
8 //   - sys_clk_period: The period of the generated clock in ns.
9 // Outputs:
10 //   - sys_clk: The system clock.
11 //   - sys_rst_n: The active-low reset signal.
12 // =====
13 module sys_signal #(
14     parameter int sys_clk_period = 10 // ns
15 ) (
16     output logic sys_clk,
17     output logic sys_rst_n
18 );
19
20 // Clock generation
21 initial begin
22     sys_clk = 1'b0;
23     forever #(sys_clk_period/2) sys_clk = ~sys_clk;
24 end
25
26 // Reset generation: hold reset low for a few cycles, then release it
27 initial begin
28     sys_rst_n = 1'b0;
29     #(sys_clk_period * 5);
30     sys_rst_n = 1'b1;
31 end
32
33 endmodule

```

```

36 // =====
37 // Module: cnt_clk_in_gen
38 // Description:
39 //   - Generates 4 divided clocks from sys_clk:
40 //     clk_div2, clk_div4, clk_div8, clk_div16.
41 //   - These serve as the 4 CLK_IN sources for IP_TIMER.
42 // =====
43 module cnt_clk_in_gen (
44     input logic sys_clk,
45     output logic [3:0] clk_in
46 );
47
48     logic clk_div2, clk_div4, clk_div8, clk_div16;
49
50     // Initialize internal divided clocks
51     initial begin
52         clk_div2 = 1'b0;
53         clk_div4 = 1'b0;
54         clk_div8 = 1'b0;
55         clk_div16 = 1'b0;
56     end
57
58     // Generate divided clocks (testbench -> use always, not always_ff)
59     always @(posedge sys_clk) clk_div2 <= ~clk_div2;
60     always @(posedge clk_div2) clk_div4 <= ~clk_div4;
61     always @(posedge clk_div4) clk_div8 <= ~clk_div8;
62     always @(posedge clk_div8) clk_div16 <= ~clk_div16;
63
64     // Map divided clocks to the output bus
65     assign clk_in = {clk_div16, clk_div8, clk_div4, clk_div2};
66
67 endmodule

```

cnt_sys_signal – clock & reset generator

```
71 // =====
72 // Module: cnt_sys_signal
73 // Description:
74 //   - Wrapper for the testbench: generates sys_clk, sys_rst_n,
75 //     and the 4 clk_in signals.
76 // =====
77 module cnt_sys_signal #(
78     parameter int sys_clk_period = 10    // ns
79 ) (
80     output logic      sys_clk_w,
81     output logic      sys_rst_n_w,
82     output logic [3:0] clk_in_w
83 );
84
85     logic sys_clk_int;
86     logic sys_rst_n_int;
87
88     // Main clock + reset
89     sys_signal #(
90         .sys_clk_period(sys_clk_period)
91     ) u_sys_signal (
92         .sys_clk  (sys_clk_int),
93         .sys_rst_n (sys_rst_n_int)
94     );
95
96     // Divided clocks
97     cnt_clk_in_gen u_cnt_clk_in_gen (
98         .sys_clk (sys_clk_int),
99         .clk_in  (clk_in_w)
100    );
101
102    // Drive testbench-visible signals
103    assign sys_clk_w  = sys_clk_int;
104    assign sys_rst_n_w = sys_rst_n_int;
105
106 endmodule
```


Test file (Test)	Category / Main goal	Example testcases inside (concept)
test_tmr_reg.sv	Register tests – verify APB register map	TDR read/write; TCR read/write with write-mask; TSR status bits; TCNT read-only; null address; mixed valid/invalid addresses (PSLVERR handling).
test_tmr_func_basic.sv	Basic counting functions	Single up-count from TDR with OVF flag set/clear; single down-count with UDF flag set/clear, repeated for all 4 CLK_IN selections.
test_tmr_func_pause.sv	Pause / Resume behaviour	Start timer (up or down), let it run, then clear EN to pause; check TCNT is almost constant during pause; set EN again and verify counter continues correctly (for all CLK_IN modes).
test_tmr_fake_udf_ovf.sv	Fake underflow / overflow injection	With timer disabled: force specific values into TCNT/TDR to emulate boundary conditions and check that TSR[OVF] / TSR[UDF] flags are asserted as specified.
test_tmr_rst_pclk2.sv	Reset while counting (PCLK×2 clock)	Start timer in up-count, assert reset, then re-start in down-count (and vice versa); verify TDR/TCR/TSR return to default values and counting restarts correctly with internal clock = PCLK×2.
test_tmr_rst_t_load_pclk2.sv	Reset + reload new value (PCLK×2 clock)	Run timer, assert reset, then load a new TDR value and start counting again; check that the new value is correctly loaded and timer counts from the new start value with PCLK×2 clock.

3.4. Testbench top

```
11 module tb_ip_TIMER;
12     localparam ADDR_WIDTH = 8;
```

```
13
14     // -----
15     // Clock / Reset / CLK_IN
16     // -----
17     logic        PCLK;
18     logic        PRESETn;
19     logic [3:0] CLK_IN;
```

```
20
21     // DUT flags
22     logic TMR_OVF;
23     logic TMR_UDF;
```

```
24
25     // var used in test
26     integer      err_cnt;
27     logic [`DATA_WIDTH-1:0] w_rand_data;
28     logic [`DATA_WIDTH-1:0] data_write;
29     logic [`DATA_WIDTH-1:0] data_read;
30     logic [`DATA_WIDTH-1:0] cnt_before_write;
31     logic [`DATA_WIDTH-1:0] cnt_after_write;
32     logic [ADDR_WIDTH-1:0] null_addr;
33     logic [ADDR_WIDTH-1:0] mixed_addr;
```

```
34
35     // -----
36     // 1. create sys_clk, sys_rst, CLK_IN
37     // -----
```

```
38     cnt_sys_signal #(
39         .sys_clk_period (10)
40     ) u_cnt_sys_signal (
41         .sys_clk_w   (PCLK),
42         .sys_rst_n_w (PRESETn),
43         .clk_in_w    (CLK_IN)
44     );
```

```
45
46     // -----
47     // 2. APB interface
48     // -----
```

```
49     apb_if #(
50         .ADDR_WIDTH(ADDR_WIDTH),
51         .DATA_WIDTH(`DATA_WIDTH)
52     ) apb (
53         .PCLK   (PCLK),
54         .PRESETn(PRESETn)
55     );
```

```
57     // Aliases for easier use in tests
58     wire          PREADY  = apb.PREADY;
59     wire          PSLVERR = apb.PSLVERR;
60     wire [`DATA_WIDTH-1:0] PRDATA = apb.PRDATA;
```

```
61
62     // -----
63     // 3. DUT: modport SLAVE
64     // -----
```

```
65     ip_TIMER #(
66         .ADDR_WIDTH(ADDR_WIDTH)
67     ) u_ip_timer (
68         .CLK_IN (CLK_IN),
69         .apb     (apb),
70         .TMR_OVF (TMR_OVF),
71         .TMR_UDF (TMR_UDF)
72     );
```

```
73
74     // -----
75     // 4. APB BFM: modport MASTER
76     // -----
```

```
77     APB_trans_bus #(
78         .ADDR_WIDTH(ADDR_WIDTH)
79     ) u_APB_trans_bus (
80         .apb(apb)
81     );
```

```
82
83     // -----
84     // 5. Include all testcase
85     // -----
```

```
86     // Register tests
87     `include "TestCase/test_tmr_reg.sv"
```

```
88
89     // Functionality tests (up/down, fork-join, pause/resume)
```

```
90     // `include "TestCase/tc_timer_func.sv"
91     `include "TestCase/test_tmr_func_basic.sv"
92     `include "TestCase/test_tmr_func_forkjoin.sv"
93     `include "TestCase/test_tmr_func_pause.sv"
```

```
94     // Reset sequence tests
95     `include "TestCase/test_tmr_rst_plck2.sv"
96     // Reset to Load sequence tests
97     `include "TestCase/test_tmr_rst_t_load_plck2.sv"
98     // Fake underflow / overflow tests
99     `include "TestCase/test_tmr_fake_udf_ovf.sv"
```


3.4. Testbench top

```
101 // -----  
102 // 6. Test controller – choose TEST=REG / FUNC / ALL  
103 // -----  
104 string TESTNAME;  
105  
106 initial begin  
107     err_cnt = 0;  
108     $display("===== START tb_ip_TIMER =====");  
109  
110     // Đợi reset nhà  
111     @(negedge PRESETn);  
112     @(posedge PRESETn);  
113     repeat (5) @(posedge PCLK);  
114  
115     // Lấy TEST từ command line (vd: +TEST=FUNC)  
116     if (!$value$plusargs("TEST=%s", TESTNAME))  
117         TESTNAME = "ALL";  
118  
119     $display("[TB] Selected TEST = %s", TESTNAME);  
120  
121     case (TESTNAME)  
122         "REG": begin  
123             run_reg_tests();  
124         end  
125  
126         "FUNC_BASIC": begin  
127             run_func_basic();  
128         end  
129  
130         "FUNC_FORKJOIN": begin  
131             run_func_forkjoin();  
132         end  
133  
134         "FUNC_PAUSE": begin  
135             run_func_pause();  
136         end  
137  
138         "FUNC_RESET": begin  
139             run_rst();  
140         end  
141  
142         "FUNC_LOAD": begin  
143             run_rst_t_load_tests();  
144         end
```

```
146 "FUNC_FAKE": begin  
147     tc_fake_underflow();  
148     tc_fake_overflow();  
149 end  
150  
151 "FUNC": begin  
152     // ----- FUNC_BASIC -----  
153     run_func_basic();  
154  
155     // ----- FUNC_FORKJOIN -----  
156     run_func_forkjoin();  
157  
158     // ----- FUNC_PAUSE -----  
159     run_func_pause();  
160  
161     // ----- Reset -----  
162     run_rst();  
163  
164     // ----- Reset+Load -----  
165     run_rst_t_load_tests();  
166  
167     // ----- Fake Underflow / Overflow -----  
168     run_fake_tests();  
169  
170 end  
171  
172 "ALL": begin  
173     // REG  
174     run_reg_tests();  
175  
176     // FUNC  
177     run_func_basic();  
178     run_func_forkjoin();  
179     run_func_pause();  
180  
181     run_rst();  
182     run_rst_t_load_tests();  
183  
184     run_fake_tests();  
185 end  
186  
187 default: begin  
188     $error("[TB] Unknown TEST=%s". TESTNAME);
```



4. Simulation Results using Makefile


```
meynchan@MEYNCHAN: ~/Desktop/Single-8Bit-Timer-main/MAKEFILE_COVERAGE$ make build
mkdir -p log
touch run_test.vt
vlib work
vmap work work
Questa Intel Starter FPGA Edition-64 vmap 2023.4 Lib Mapping Utility 2023.10 Oct 9 2023
vmap work work
Copying /home/meynchan/questasim/questa_fse/linux_x86_64/./modelsim.ini to modelsim.ini
Modifying modelsim.ini
# Compile có coverage + visibility tốt để debug
# compile.f: +incdir+rtl +incdir+tb +incdir+tb/VIP +incdir+tb/TestCase + tb/tb_ip_TIMER.sv
vlog -coveropt 3 +cover +acc +access+rcw -f compile.f
Questa Intel Starter FPGA Edition-64 vlog 2023.4 Compiler 2023.10 Oct 9 2023
Start time: 03:08:26 on Dec 10,2025
vlog -coveropt 3 "+cover" "+acc" "+access+rcw" -f compile.f
-- Compiling interface apb_if
-- Compiling module sys_signal
-- Compiling module cnt_clk_in_gen
-- Compiling module cnt_sys_signal
-- Compiling module APB_trans_bus
-- Compiling module rw_reg_control
-- Compiling module rw_read_logic
-- Compiling module rw_write_logic
-- Compiling module APB_trans
-- Compiling module pos_cnt_edge_detect
-- Compiling module pos_edge_detect_synced
-- Compiling module clk_sel_synced_alt
-- Compiling module logic_control
-- Compiling module cnt_unit
-- Compiling module ovf_udf_comp
-- Compiling module ip_TIMER
-- Compiling module tb_ip_TIMER

Top level modules:
    tb_ip_TIMER
End time: 03:08:26 on Dec 10,2025, Elapsed time: 0:00:00
Errors: 0, Warnings: 0
meynchan@MEYNCHAN: ~/Desktop/Single-8Bit-Timer-main/MAKEFILE_COVERAGE$ make run TESTNAME=ALL
# Dùng -voptargs="+acc" cho visibility; tránh +acc+rcw, -access, -novopt ở đây
vsim -l ALL.log -voptargs="+acc" -assertdebug \
    -c tb_ip_TIMER \
    +TEST=ALL \
```

- Makefile targets
 - + make reg → Register tests
 - + make func → Functional tests (up/down, pause, reset, ...)
 - + make all → Run all tests + generate log
- One command = run testbench + compile + run + save log
- TEST name passed by plusargs: +TEST=REG / FUNC / ALL

4. Simulation Results using Makefile

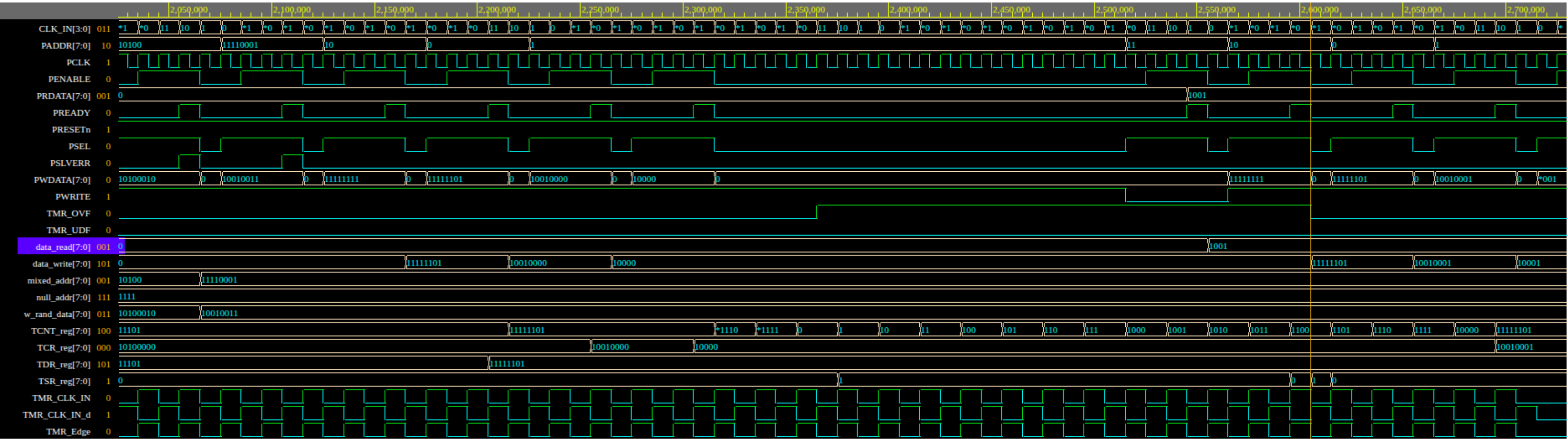
meynchan@MEYNCHAN: ~/Desktop/Single-8Bit-Timer-main/MAKEFILE_COVERAGE

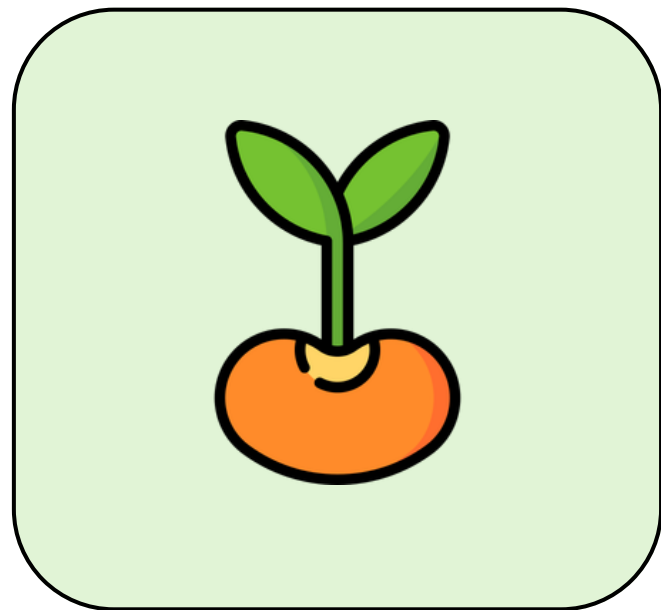
```
# ===== START tb_ip_TIMER =====
# [TB] Selected TEST = ALL
#
# [--- TC1.1: TDR read/write ---]
# TC1.1-1 PASS: Default TDR value = 0x00
# TC1.1-2 PASS: Wrote & read 0x9a correctly
# TC1.1-2 PASS: Wrote & read 0xb6 correctly
# TC1.1-2 PASS: Wrote & read 0xdc correctly
# TC1.1-2 PASS: Wrote & read 0x00 correctly
# TC1.1-2 PASS: Wrote & read 0x79 correctly
# TC1.1-2 PASS: Wrote & read 0x6d correctly
# TC1.1-2 PASS: Wrote & read 0xcc correctly
# TC1.1-2 PASS: Wrote & read 0x9d correctly
# TC1.1-2 PASS: Wrote & read 0x48 correctly
# TC1.1-2 PASS: Wrote & read 0x91 correctly
# TC1.1-2 PASS: Wrote & read 0x16 correctly
# TC1.1-2 PASS: Wrote & read 0x86 correctly
# TC1.1-2 PASS: Wrote & read 0x14 correctly
# TC1.1-2 PASS: Wrote & read 0xa3 correctly
# TC1.1-2 PASS: Wrote & read 0x6d correctly
# TC1.1-2 PASS: Wrote & read 0x11 correctly
# TC1.1-2 PASS: Wrote & read 0xa1 correctly
# TC1.1-2 PASS: Wrote & read 0x65 correctly
# TC1.1-2 PASS: Wrote & read 0x4c correctly
# TC1.1-2 PASS: Wrote & read 0x60 correctly
#
# [--- TC1.2: TCR read/write mask ---]
# TC1.2-1 PASS: Default TCR value = 0x00
# TC1.2-2 PASS: Wrote 0x42, read masked 0x02 OK
# TC1.2-2 PASS: Wrote 0xc4, read masked 0x80 OK
# TC1.2-2 PASS: Wrote 0x02, read masked 0x02 OK
# TC1.2-2 PASS: Wrote 0x32, read masked 0x32 OK
# TC1.2-2 PASS: Wrote 0x70, read masked 0x30 OK
# TC1.2-2 PASS: Wrote 0x37, read masked 0x33 OK
# TC1.2-2 PASS: Wrote 0x88, read masked 0x80 OK
# TC1.2-2 PASS: Wrote 0x97, read masked 0x93 OK
# TC1.2-2 PASS: Wrote 0x9f, read masked 0x93 OK
# TC1.2-2 PASS: Wrote 0xa4, read masked 0xa0 OK
# TC1.2-2 PASS: Wrote 0x82, read masked 0x82 OK
# TC1.2-2 PASS: Wrote 0x71, read masked 0x31 OK
# TC1.2-2 PASS: Wrote 0x72, read masked 0x32 OK
# TC1.2-2 PASS: Wrote 0x63, read masked 0x23 OK
# TC1.2-2 PASS: Wrote 0x44, read masked 0x00 OK
# TC1.2-2 PASS: Wrote 0x05, read masked 0x01 OK
# TC1.2-2 PASS: Wrote 0xd4, read masked 0x90 OK
# TC1.2-2 PASS: Wrote 0x55, read masked 0x11 OK
# TC1.2-2 PASS: Wrote 0x9c, read masked 0x90 OK
```

meynchan@MEYNCHAN: ~/Desktop/Single-8Bit-Timer-main/MAKEFILE_COVERAGE

```
# TC2.5-2 PASS: TCNT changes after resume (0x84 -> 0x79)
# TC2.5: PASS
#
# [--- TC2.5: DOWN Pause/Resume, CKS=3 ---]
# TCNT before pause = 0x70
# TC2.5-1 PASS: TCNT is almost stable while paused (0x70 -> 0x70, diff=0)
# TC2.5-2 PASS: TCNT changes after resume (0x70 -> 0x66)
# TC2.5: PASS
#
# [TC2.6] countdw_reset_countup_pclk2
# Before reset: TCNT = 0x59
# Reset part PASS: TDR/TCR/TSR back to default
# Up-count part PASS: OVF flag set (TSR=0x1)
# TC2.6 countdw_reset_countup_pclk2: PASS
#
# [TC2.7] countup_reset_countdw_pclk2
# Before reset: TCNT = 0xb6
# Reset part PASS: TDR/TCR/TSR back to default
# Down-count part PASS: UDF flag set (TSR=0x2)
# TC2.7 countup_reset_countdw_pclk2: PASS
#
# [TC2.8] countup_reset_load_countdw_pclk2
# Reset part PASS: TDR/TCR/TSR back to default
# Load+Down part PASS: UDF flag set (TSR=0x2)
# TC2.8 countup_reset_load_countdw_pclk2: PASS
#
# [TC2.9] countdw_reset_load_countdw_pclk2
# Reset part PASS: TDR/TCR/TSR back to default
# Load+Down part PASS: UDF flag set (TSR=0x2)
# TC2.9 countdw_reset_load_countdw_pclk2: PASS
#
# [--- TC2.10: fake_underflow ---]
# TC2.10 PASS: UDF flag NOT set in fake_underflow (TSR=0x0)
#
# [--- TC2.11: fake_overflow ---]
# TC2.11 PASS: OVF flag NOT set in fake_overflow (TSR=0x0)
# ===== END tb_ip_TIMER, err_cnt = 0 =====
# ** Note: $finish      : tb/tb_ip_TIMER.sv(199)
# Time: 96395 ns Iteration: 0 Instance: /tb_ip_TIMER
# End time: 21:21:28 on Dec 09,2025, Elapsed time: 0:00:01
# Errors: 0, Warnings: 1
mv ALL.log ./log
cp -rf vsim.wlf ALL.wlf
mv ALL.wlf ./log
ln -sf ./log/ALL.log sim.log
meynchan@MEYNCHAN:~/Desktop/Single-8Bit-Timer-main/MAKEFILE_COVERAGE$
```

4. Simulation Results using Makefile





100%

Thanks for listening